Etape 1 : Concaténer tous les fichiers mensuels en une seule base

🎯 Objectif

Construire automatiquement **une base unique** à partir des fichiers mensuels CNPS enregistrés dans `./CNPS_FUSION/MENSUELS`, en :

- listant les fichiers disponibles,
- chargeant les fichiers,
- les concaténant en une seule base,
- sauvegardant le résultat dans un format adapté (Excel ou Parquet+CSV si très volumineux),
- fournissant des outils d'analyse et de diagnostics.

🧩 Les différentes étapes

1. `lister_fichiers_mensuels()`

Repère tous les fichiers mensuels CNPS correspondant au pattern `TRAVAILLEURS_YYYY_MM_NUMERO_CNPS.xlsx`.

- Vérifie que le dossier existe.
- Trie les fichiers trouvés.
- Affiche leurs tailles.

Rôle : préparer la liste des fichiers à concaténer.

---

2. `charger_excel_local()`

Charge un fichier Excel et affiche :

- nombre de lignes,
- nombre de colonnes.

Rôle : chargement robuste, centralisé, avec logs.

---

3. `sauvegarder_excel_local()`

Sauvegarde le DataFrame final :

- Si ≤ 1 048 576 lignes → **Excel**.
- Si > limite Excel → **Parquet + CSV** automatiquement.

Rôle : éviter les erreurs Excel et adapter automatiquement le format.

---

4. `analyser_fichiers()`

Analyse rapide fichier par fichier :

- période extraite du nom,
- nombre de lignes et colonnes,
- taille en MB.

Rôle : diagnostic avant concaténation.

---

5. `verifier_coherence()`

Contrôle si les fichiers ont :

- les **mêmes colonnes**,
- les **mêmes types** de variables.

Rôle : s'assurer que la concaténation ne va pas créer de problèmes.

---

6. `analyser_concatenation()`

Analyse la base finale :

- nombre total de lignes / colonnes,
- mémoire en MB,
- duplications éventuelles sur `NUMERO_CNPS`,
- distribution par période (si disponible).

Rôle : contrôle qualité post-concaténation.

---

7. `main()`

Pipeline complet :

1. Liste les fichiers.
2. Ne garde que les **nb_fichiers_max** premiers (par défaut 3).
3. Charge chaque fichier.
4. Concatène.
5. Sauvegarde au bon format.
6. Loggue un résumé final.

Rôle : bouton principal pour créer la base concaténée.

---

8. `sauvegarder_en_plusieurs_excel()`

Scinde un très gros DataFrame en plusieurs fichiers Excel (ex. blocs de 1 000 000 lignes).

Rôle : produire des fichiers Excel exploitables malgré la limite d'Excel.

---

9. `concatener_periode_specifique()`

Concatène uniquement les fichiers compris entre deux périodes (ex. janv. 2024 → juin 2024).

Rôle : extraction ciblée d'une période.

---

10. Bloc final

Exécute automatiquement :

```
df_concat = main(nb_fichiers_max=3)
```

Rôle : mode test : concaténer seulement 3 fichiers.

```
import pandas as pd
import glob
import os
import logging
from pathlib import Path
from datetime import datetime
from typing import List, Optional

# ============================================================================
# CONFIGURATION ET LOGGING
# ============================================================================

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)
logger = logging.getLogger(__name__)

# Configuration des chemins
DOSSIER_MENSUELS = "./CNPS_FUSION/MENSUELS"
DOSSIER_SORTIE = "./CNPS_FUSION"
NOM_FICHIER_SORTIE = "TRAVAILLEURS_CONCATENES_2024_2025.xlsx"

# ============================================================================
# FONCTIONS DE LISTING ET CHARGEMENT
```

```python
# ==============================================================================

def lister_fichiers_mensuels(dossier: str = DOSSIER_MENSUELS) -> List[str]:
    """
    Liste tous les fichiers mensuels dans le dossier local

    Args:
        dossier: Chemin du dossier contenant les fichiers mensuels

    Returns:
        Liste des chemins de fichiers trouvés
    """
    logger.info(f"📂 Recherche des fichiers dans : {dossier}")

    if not Path(dossier).exists():
        logger.error(f"❌ Le dossier {dossier} n'existe pas")
        return []

    # Pattern pour trouver les fichiers mensuels
    pattern = os.path.join(dossier, "TRAVAILLEURS_*_*_NUMERO_CNPS.xlsx")
    fichiers = sorted(glob.glob(pattern))

    logger.info(f"✓ {len(fichiers)} fichiers mensuels trouvés")

    if fichiers:
        logger.info("Liste des fichiers :")
        for f in fichiers:
            nom = Path(f).name
            taille_mb = Path(f).stat().st_size / (1024 * 1024)
            logger.info(f"  - {nom:50s} ({taille_mb:6.2f} MB)")

    return fichiers


def charger_excel_local(chemin_fichier: str) -> pd.DataFrame:
    """
    Charge un fichier Excel depuis le système de fichiers local

    Args:
        chemin_fichier: Chemin du fichier à charger

    Returns:
        DataFrame chargé
    """
    try:
        logger.info(f"📥 Chargement : {Path(chemin_fichier).name}")
        df = pd.read_excel(chemin_fichier, engine='openpyxl')
        logger.info(f"   ✓ {df.shape[0]:,} lignes × {df.shape[1]} colonnes")
        return df

    except Exception as e:
        logger.error(f"❌ Erreur lors du chargement de {chemin_fichier}: {e}")
        raise
```

```python
def sauvegarder_excel_local(
    df: pd.DataFrame,
    nom_fichier: str,
    dossier: str = DOSSIER_SORTIE,
    auto_format: bool = True
) -> str:
    """
    Sauvegarde un DataFrame dans le format approprié selon la taille

    Excel a une limite de 1 048 576 lignes. Si dépassement, sauvegarde en Parquet
+ CSV.

    Args:
        df: DataFrame à sauvegarder
        nom_fichier: Nom du fichier de sortie
        dossier: Dossier de destination
        auto_format: Si True, choisit automatiquement le format selon la taille

    Returns:
        Chemin complet du fichier sauvegardé
    """
    try:
        # Créer le dossier si nécessaire
        Path(dossier).mkdir(parents=True, exist_ok=True)

        # Limite Excel
        EXCEL_MAX_ROWS = 1_048_576
        n_lignes = len(df)

        taille_estimee_mb = df.memory_usage(deep=True).sum() / (1024 * 1024)

        # Vérifier si le fichier dépasse la limite Excel
        if auto_format and n_lignes > EXCEL_MAX_ROWS:
            logger.warning(f"⚠️ ATTENTION : {n_lignes:,} lignes > limite Excel
({EXCEL_MAX_ROWS:,})")
            logger.info(f"   → Sauvegarde automatique en PARQUET + CSV")

            # Sauvegarder en Parquet (optimal pour gros volumes)
            nom_base = nom_fichier.replace('.xlsx', '')
            chemin_parquet = os.path.join(dossier, f"{nom_base}.parquet")
            chemin_csv = os.path.join(dossier, f"{nom_base}.csv")

            logger.info(f"💾 Sauvegarde PARQUET : {chemin_parquet} (~
{taille_estimee_mb:.2f} MB)")
            df.to_parquet(chemin_parquet, index=False, compression='snappy')
            taille_parquet = Path(chemin_parquet).stat().st_size / (1024 * 1024)
            logger.info(f"   ✅ Parquet sauvegardé : {taille_parquet:.2f} MB")

            logger.info(f"💾 Sauvegarde CSV : {chemin_csv}")
            df.to_csv(chemin_csv, index=False, encoding='utf-8-sig')
            taille_csv = Path(chemin_csv).stat().st_size / (1024 * 1024)
            logger.info(f"   ✅ CSV sauvegardé : {taille_csv:.2f} MB")
```

```
            logger.info(f"\n📊 Récapitulatif des fichiers :")
            logger.info(f"   - PARQUET (recommandé) : {chemin_parquet}")
            logger.info(f"   - CSV (compatible)     : {chemin_csv}")

            return chemin_parquet  # Retourne le chemin Parquet comme référence

        else:
            # Sauvegarde Excel normale
            chemin_complet = os.path.join(dossier, nom_fichier)
            logger.info(f"💾 Sauvegarde EXCEL : {chemin_complet} (~
{taille_estimee_mb:.2f} MB)")

            df.to_excel(chemin_complet, index=False, engine='openpyxl')

            taille_reelle_mb = Path(chemin_complet).stat().st_size / (1024 * 1024)
            logger.info(f"   ✅ Excel sauvegardé : {taille_reelle_mb:.2f} MB")

            return chemin_complet

    except Exception as e:
        logger.error(f"❌ Erreur lors de la sauvegarde: {e}")
        raise


# ============================================================================
# FONCTIONS D'ANALYSE
# ============================================================================

def analyser_fichiers(fichiers: List[str]) -> pd.DataFrame:
    """
    Analyse les fichiers mensuels sans les charger complètement

    Args:
        fichiers: Liste des chemins de fichiers

    Returns:
        DataFrame avec les statistiques par fichier
    """
    logger.info("\n📊 Analyse des fichiers mensuels...")

    stats = []
    for fichier in fichiers:
        nom = Path(fichier).name

        # Extraire période du nom
        parts = nom.replace(".xlsx", "").split("_")
        if len(parts) >= 3:
            annee = parts[1]
            mois = parts[2]
            periode = f"{annee}-{mois}"
        else:
            periode = "Inconnu"

        # Taille du fichier
```

```
        taille_mb = Path(fichier).stat().st_size / (1024 * 1024)

        # Charger juste pour compter
        df_temp = pd.read_excel(fichier, engine='openpyxl')
        n_lignes = len(df_temp)
        n_colonnes = len(df_temp.columns)

        stats.append({
            "Fichier": nom,
            "Période": periode,
            "Lignes": n_lignes,
            "Colonnes": n_colonnes,
            "Taille_MB": round(taille_mb, 2)
        })

    df_stats = pd.DataFrame(stats)

    logger.info("\nRésumé par fichier :")
    logger.info("\n" + df_stats.to_string(index=False))

    return df_stats


def verifier_coherence(df_list: List[pd.DataFrame]) -> dict:
    """
    Vérifie la cohérence entre les DataFrames avant concaténation

    Args:
        df_list: Liste des DataFrames à vérifier

    Returns:
        Dictionnaire avec les résultats de vérification
    """
    logger.info("\n🔍 Vérification de la cohérence des fichiers...")

    # Colonnes
    colonnes_ref = set(df_list[0].columns)
    colonnes_identiques = all(set(df.columns) == colonnes_ref for df in df_list)

    if colonnes_identiques:
        logger.info("   ✅ Toutes les colonnes sont identiques")
    else:
        logger.warning("   ⚠️ Les colonnes diffèrent entre les fichiers")
        for i, df in enumerate(df_list):
            cols_diff = set(df.columns).symmetric_difference(colonnes_ref)
            if cols_diff:
                logger.warning(f"      Fichier {i+1}: différences = {cols_diff}")

    # Types de données
    types_ref = df_list[0].dtypes.to_dict()
    types_identiques = True

    for i, df in enumerate(df_list[1:], start=2):
        types_df = df.dtypes.to_dict()
```

```
        for col in colonnes_ref:
            if col in types_df and types_ref.get(col) != types_df[col]:
                types_identiques = False
                logger.warning(
                    f"   ⚠ Type différent pour '{col}' dans fichier {i}: "
                    f"{types_ref[col]} vs {types_df[col]}"
                )

    if types_identiques:
        logger.info("   ✅ Types de données cohérents")

    return {
        "colonnes_identiques": colonnes_identiques,
        "types_identiques": types_identiques,
        "nombre_colonnes": len(colonnes_ref),
        "colonnes": list(colonnes_ref)
    }


def analyser_concatenation(df_concat: pd.DataFrame) -> dict:
    """
    Analyse le DataFrame concaténé

    Args:
        df_concat: DataFrame concaténé

    Returns:
        Dictionnaire avec les statistiques
    """
    logger.info("\n📊 Analyse du DataFrame concaténé...")

    stats = {
        "total_lignes": len(df_concat),
        "total_colonnes": len(df_concat.columns),
        "taille_memoire_mb": df_concat.memory_usage(deep=True).sum() / (1024 *
1024)
    }

    # Vérifications si NUMERO_CNPS existe
    if "NUMERO_CNPS" in df_concat.columns:
        n_uniques = df_concat["NUMERO_CNPS"].nunique()
        n_duplications = len(df_concat) - n_uniques

        stats["numero_cnps_uniques"] = n_uniques
        stats["duplications"] = n_duplications

        logger.info(f"   Total de lignes        : {stats['total_lignes']:,}")
        logger.info(f"   NUMERO_CNPS uniques    : {n_uniques:,}")
        logger.info(f"   Duplications           : {n_duplications:,}")

        if n_duplications > 0:
            logger.warning(f"   ⚠ {n_duplications:,} NUMERO_CNPS dupliqués
détectés")
        else:
```

```
            logger.info("   ✅ Tous les NUMERO_CNPS sont uniques")

    # Distribution par période si la colonne existe
    if "PERIODE" in df_concat.columns:
        dist_periode = df_concat["PERIODE"].value_counts().sort_index()
        logger.info("\n   Distribution par période :")
        for periode, count in dist_periode.items():
            logger.info(f"      {periode}: {count:,} lignes")
        stats["periodes"] = dist_periode.to_dict()

    logger.info(f"\n   Mémoire utilisée       : {stats['taille_memoire_mb']:.2f}
MB")

    return stats


# ============================================================================
# FONCTION PRINCIPALE
# ============================================================================
def main(
    dossier_mensuels: str = DOSSIER_MENSUELS,
    nom_sortie: str = NOM_FICHIER_SORTIE,
    avec_analyse: bool = True,
    sauvegarder_stats: bool = True,
    nb_fichiers_max: int = 3  # ◇ nouveau paramètre : nombre de fichiers à
charger
) -> Optional[pd.DataFrame]:
    """
    Pipeline principal de concaténation

    Args:
        dossier_mensuels: Dossier contenant les fichiers mensuels
        nom_sortie: Nom du fichier de sortie
        avec_analyse: Si True, effectue des analyses détaillées
        sauvegarder_stats: Si True, sauvegarde les statistiques
        nb_fichiers_max: Nombre maximum de fichiers à charger et concaténer
    """
    import time
    start_time = time.time()

    logger.info("=" * 80)
    logger.info("🚀 CONCATÉNATION DES FICHIERS MENSUELS CNPS (VERSION LOCALE)")
    logger.info("=" * 80)

    try:
        # ÉTAPE 1: Listing des fichiers
        logger.info("\n📁 ÉTAPE 1 : LISTING DES FICHIERS")
        logger.info("-" * 80)
        fichiers = lister_fichiers_mensuels(dossier_mensuels)

        if not fichiers:
            logger.error("⚠️ Aucun fichier mensuel trouvé.")
            logger.info("💡 Assurez-vous d'avoir exécuté le script de
génération")
```

```
            return None

        #  ◇  Ne garder que nb_fichiers_max fichiers
        if nb_fichiers_max is not None and nb_fichiers_max > 0:
            fichiers = fichiers[:nb_fichiers_max]
            logger.info(f"📌 Seuls les {len(fichiers)} premiers fichiers seront
concaténés.")



        # ÉTAPE 3: Chargement des fichiers sélectionnés
        logger.info("\n📥 ÉTAPE 3 : CHARGEMENT DES FICHIERS SÉLECTIONNÉS")
        logger.info("-" * 80)
        df_list = []

        for i, fichier in enumerate(fichiers, 1):
            logger.info(f"[{i}/{len(fichiers)}] Chargement en cours...")
            df_mois = charger_excel_local(fichier)
            df_list.append(df_mois)

        # Vérification minimale
        if not df_list:
            logger.error("⚠️ Aucun DataFrame chargé, arrêt.")
            return None



        # ÉTAPE 5: Concaténation
        logger.info("\n🔗 ÉTAPE 5 : CONCATÉNATION")
        logger.info("-" * 80)
        logger.info("   Fusion des DataFrames sélectionnés...")
        df_concat = pd.concat(df_list, ignore_index=True)
        logger.info(f"   ✓ Base concaténée : {df_concat.shape[0]:,} lignes ×
{df_concat.shape[1]} colonnes")



        # ÉTAPE 7: Sauvegarde
        logger.info("\n💾 ÉTAPE 7 : SAUVEGARDE")
        logger.info("-" * 80)
        chemin_sortie = sauvegarder_excel_local(df_concat, nom_sortie)
        logger.info(f"   📍 Fichier disponible : {chemin_sortie}")

        # Résumé final
        elapsed_time = time.time() - start_time
        logger.info("\n" + "=" * 80)
        logger.info("✅ CONCATÉNATION TERMINÉE AVEC SUCCÈS")
        logger.info(f"⏱️  Temps d'exécution: {elapsed_time:.2f} secondes
({elapsed_time/60:.1f} min)")
        logger.info(f"📊 Résultat:")
        logger.info(f"   - Fichiers traités : {len(fichiers)}")
        logger.info(f"   - Lignes totales   : {len(df_concat):,}")
        logger.info(f"   - Colonnes         : {len(df_concat.columns)}")
        logger.info(f"   - Fichier de sortie: {chemin_sortie}")
```

```
        logger.info("=" * 80)

        return df_concat

    except Exception as e:
        logger.error(f"\n❌ ERREUR CRITIQUE: {e}", exc_info=True)
        raise



# ============================================================================
# FONCTIONS UTILITAIRES SUPPLÉMENTAIRES
# ============================================================================

def sauvegarder_en_plusieurs_excel(
    df: pd.DataFrame,
    nom_base: str = "TRAVAILLEURS_CONCATENES",
    dossier: str = DOSSIER_SORTIE,
    lignes_par_fichier: int = 1_000_000
) -> List[str]:
    """
    Divise un gros DataFrame en plusieurs fichiers Excel

    Args:
        df: DataFrame à diviser
        nom_base: Nom de base des fichiers
        dossier: Dossier de destination
        lignes_par_fichier: Nombre de lignes par fichier (max 1 048 576)

    Returns:
        Liste des chemins des fichiers créés
    """
    logger.info(f"\n📑 Division en fichiers Excel multiples...")
    logger.info(f"   Total de lignes : {len(df):,}")
    logger.info(f"   Lignes par fichier : {lignes_par_fichier:,}")

    n_fichiers = (len(df) + lignes_par_fichier - 1) // lignes_par_fichier
    logger.info(f"   Nombre de fichiers : {n_fichiers}")

    fichiers_crees = []

    for i in range(n_fichiers):
        debut = i * lignes_par_fichier
        fin = min((i + 1) * lignes_par_fichier, len(df))

        df_part = df.iloc[debut:fin]
        nom_fichier = f"{nom_base}_partie_{i+1:02d}_sur_{n_fichiers:02d}.xlsx"

        logger.info(f"\n   Partie {i+1}/{n_fichiers} : lignes {debut:,} à {fin-
1:,}")
        chemin = sauvegarder_excel_local(
            df_part,
            nom_fichier,
            dossier,
```

```
            auto_format=False  # Forcer Excel
        )
        fichiers_crees.append(chemin)

    logger.info(f"\n✅ {n_fichiers} fichiers Excel créés")
    return fichiers_crees


def concatener_periode_specifique(
    annee_debut: int,
    mois_debut: int,
    annee_fin: int,
    mois_fin: int,
    nom_sortie: Optional[str] = None
) -> pd.DataFrame:
    """
    Concatène uniquement les fichiers d'une période spécifique

    Args:
        annee_debut: Année de début
        mois_debut: Mois de début (1-12)
        annee_fin: Année de fin
        mois_fin: Mois de fin (1-12)
        nom_sortie: Nom du fichier de sortie (optionnel)

    Returns:
        DataFrame concaténé
    """
    logger.info(f"📅 Concaténation de {annee_debut}-{mois_debut:02d} à
{annee_fin}-{mois_fin:02d}")

    # Lister tous les fichiers
    tous_fichiers = lister_fichiers_mensuels()

    # Filtrer par période
    fichiers_periode = []
    for fichier in tous_fichiers:
        nom = Path(fichier).name
        parts = nom.replace(".xlsx", "").split("_")

        if len(parts) >= 3:
            annee = int(parts[1])
            mois = int(parts[2])

            # Vérifier si dans la période
            date_fichier = annee * 100 + mois
            date_debut = annee_debut * 100 + mois_debut
            date_fin = annee_fin * 100 + mois_fin

            if date_debut <= date_fichier <= date_fin:
                fichiers_periode.append(fichier)

    logger.info(f"   {len(fichiers_periode)} fichiers dans la période")
```

```
    # Charger et concaténer
    df_list = [charger_excel_local(f) for f in fichiers_periode]
    df_concat = pd.concat(df_list, ignore_index=True)

    # Sauvegarder
    if nom_sortie is None:
        nom_sortie = f"TRAVAILLEURS_{annee_debut}{mois_debut:02d}_a_{annee_fin}
{mois_fin:02d}.xlsx"

    sauvegarder_excel_local(df_concat, nom_sortie)

    return df_concat


# ============================================================================
# POINT D'ENTRÉE
# ============================================================================
if __name__ == "__main__":
    # Ne concatène que les 3 premiers fichiers trouvés dans ./CNPS_FUSION/MENSUELS
    df_concat = main(nb_fichiers_max=3)
```

```
2025-12-04 15:19:13 - INFO -
================================================================================
2025-12-04 15:19:13 - INFO - 🚀 CONCATÉNATION DES FICHIERS MENSUELS CNPS (VERSION
LOCALE)
2025-12-04 15:19:13 - INFO -
================================================================================
2025-12-04 15:19:13 - INFO -
📁 ÉTAPE 1 : LISTING DES FICHIERS
2025-12-04 15:19:13 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 15:19:13 - INFO - 📂 Recherche des fichiers dans :
./CNPS_FUSION/MENSUELS
2025-12-04 15:19:13 - INFO - ✓ 24 fichiers mensuels trouvés
2025-12-04 15:19:13 - INFO - Liste des fichiers :
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_01_NUMERO_CNPS.xlsx
( 50.54 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_02_NUMERO_CNPS.xlsx
( 50.64 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_03_NUMERO_CNPS.xlsx
( 50.74 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_04_NUMERO_CNPS.xlsx
( 50.85 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_05_NUMERO_CNPS.xlsx
( 50.95 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_06_NUMERO_CNPS.xlsx
( 51.05 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_07_NUMERO_CNPS.xlsx
( 51.15 MB)
```

```
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_08_NUMERO_CNPS.xlsx
( 51.25 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_09_NUMERO_CNPS.xlsx
( 51.35 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_10_NUMERO_CNPS.xlsx
( 51.45 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_11_NUMERO_CNPS.xlsx
( 51.55 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2024_12_NUMERO_CNPS.xlsx
( 51.66 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_01_NUMERO_CNPS.xlsx
( 51.76 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_02_NUMERO_CNPS.xlsx
( 51.86 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_03_NUMERO_CNPS.xlsx
( 51.96 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_04_NUMERO_CNPS.xlsx
( 52.06 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_05_NUMERO_CNPS.xlsx
( 52.16 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_06_NUMERO_CNPS.xlsx
( 52.26 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_07_NUMERO_CNPS.xlsx
( 52.37 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_08_NUMERO_CNPS.xlsx
( 52.47 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_09_NUMERO_CNPS.xlsx
( 52.57 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_10_NUMERO_CNPS.xlsx
( 52.67 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_11_NUMERO_CNPS.xlsx
( 52.77 MB)
2025-12-04 15:19:13 - INFO -   - TRAVAILLEURS_2025_12_NUMERO_CNPS.xlsx
( 52.88 MB)
2025-12-04 15:19:13 - INFO - 📌 Seuls les 3 premiers fichiers seront concaténés.
2025-12-04 15:19:13 - INFO -
📥 ÉTAPE 3 : CHARGEMENT DES FICHIERS SÉLECTIONNÉS
2025-12-04 15:19:13 - INFO - ------------------------------------------------------
---------------------------
2025-12-04 15:19:13 - INFO - [1/3] Chargement en cours...
2025-12-04 15:19:13 - INFO - 📥 Chargement :
TRAVAILLEURS_2024_01_NUMERO_CNPS.xlsx
2025-12-04 15:21:08 - INFO -    ✓ 500,000 lignes × 14 colonnes
2025-12-04 15:21:08 - INFO - [2/3] Chargement en cours...
2025-12-04 15:21:08 - INFO - 📥 Chargement :
TRAVAILLEURS_2024_02_NUMERO_CNPS.xlsx
2025-12-04 15:23:00 - INFO -    ✓ 501,000 lignes × 14 colonnes
2025-12-04 15:23:00 - INFO - [3/3] Chargement en cours...
2025-12-04 15:23:00 - INFO - 📥 Chargement :
TRAVAILLEURS_2024_03_NUMERO_CNPS.xlsx
2025-12-04 15:24:53 - INFO -    ✓ 502,000 lignes × 14 colonnes
```

```
2025-12-04 15:24:53 - INFO -
🔗 ÉTAPE 5 : CONCATÉNATION
2025-12-04 15:24:53 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 15:24:53 - INFO -    Fusion des DataFrames sélectionnés...
2025-12-04 15:24:53 - INFO -    ✓ Base concaténée : 1,503,000 lignes × 14
colonnes
2025-12-04 15:24:53 - INFO -
💾 ÉTAPE 7 : SAUVEGARDE
2025-12-04 15:24:53 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 15:24:56 - WARNING - ⚠️ ATTENTION : 1,503,000 lignes > limite Excel
(1,048,576)
2025-12-04 15:24:56 - INFO -    → Sauvegarde automatique en PARQUET + CSV
2025-12-04 15:24:56 - INFO - 💾 Sauvegarde PARQUET :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025.parquet (~951.38 MB)
2025-12-04 15:25:00 - INFO -    ☑ Parquet sauvegardé : 42.99 MB
2025-12-04 15:25:00 - INFO - 💾 Sauvegarde CSV :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025.csv
2025-12-04 15:25:10 - INFO -    ☑ CSV sauvegardé : 289.30 MB
2025-12-04 15:25:10 - INFO -
📊 Récapitulatif des fichiers :
2025-12-04 15:25:10 - INFO -    - PARQUET (recommandé) :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025.parquet
2025-12-04 15:25:10 - INFO -    - CSV (compatible)     :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025.csv
2025-12-04 15:25:10 - INFO -    📍 Fichier disponible :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025.parquet
2025-12-04 15:25:10 - INFO -
================================================================================
2025-12-04 15:25:10 - INFO - ☑ CONCATÉNATION TERMINÉE AVEC SUCCÈS
2025-12-04 15:25:10 - INFO - ⏱  Temps d'exécution: 357.64 secondes (6.0 min)
2025-12-04 15:25:10 - INFO - 📊 Résultat:
2025-12-04 15:25:10 - INFO -    - Fichiers traités : 3
2025-12-04 15:25:10 - INFO -    - Lignes totales   : 1,503,000
2025-12-04 15:25:10 - INFO -    - Colonnes         : 14
2025-12-04 15:25:10 - INFO -    - Fichier de sortie:
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025.parquet
2025-12-04 15:25:10 - INFO -
================================================================================
```

Etape 2 : Extraire la colonne NUMERO_CNPS unique (supprimer les doublons)

🎯 Objectif

Ce script extrait **tous les NUMERO_CNPS uniques** à partir d'une base concaténée (Excel / Parquet / CSV), analyse les duplications et sauvegarde les résultats dans plusieurs formats (Excel, CSV, TXT). Il permet de

préparer une **liste propre, dédoublonnée et analysée** des numéros CNPS avant pseudonymisation ou appariement.

🧩 Les différentes étapes

---

1. Chargement des fichiers (`charger_fichier_local`)

Charge automatiquement un fichier local selon son extension :

- `.xlsx` → `read_excel`
- `.parquet` → `read_parquet`
- `.csv` → `read_csv`

Affiche aussi :

- le nombre de lignes / colonnes,
- la taille du fichier.

Rôle : fournir une fonction unique, flexible et robuste pour lire n'importe quel format.

---

2. Sauvegarde Excel (`sauvegarder_excel_local`)

Sauvegarde un DataFrame en Excel dans `./CNPS_FUSION`.

- Crée le dossier si nécessaire.
- Log la taille estimée et réelle du fichier.

Rôle : exporter les résultats sous un format lisible

---

3. Extraire les NUMERO_CNPS uniques (`extraire_numero_cnps_uniques`)

Fonction centrale du script :

1. Vérifie si la colonne existe.
2. Convertit la colonne en chaîne (string).
3. Supprime les doublons.
4. Produit des statistiques :
    - nombre total de lignes,
    - nombre d'uniques,
    - taux de duplication,
    - répartition des longueurs des numéros,
    - distribution du 1er chiffre (code sexe CNPS 1=H, 2=F).

Rôle : construire une base propre, dédoublonnée, et documentée des numéros CNPS.

---

4. Analyse détaillée des duplications (`analyser_duplications`)

Détecte :

- combien de numéros apparaissent 1 fois / plusieurs fois,
- la duplication maximale,
- la distribution des duplications,
- le top 10 des numéros les plus fréquents,
- un tableau détaillé `NUMERO_CNPS / Nombre_Occurrences`.

Rôle : comprendre l'origine des doublons et documenter les anomalies.

---

5. Sauvegarder dans plusieurs formats (`sauvegarder_formats_multiples`)

Génère automatiquement :

- un **Excel** (`.xlsx`)
- un **CSV** (`.csv`)
- un **TXT** contenant uniquement la liste des numéros (`.txt`)

Rôle : fournir des outputs adaptés aux besoins métiers, techniques et SIG.

---

6. Fonction principale `main()`

Pipeline complet :

1. Charge le fichier source (Excel / Parquet / CSV).
2. Analyse les duplications (optionnel).
3. Extrait les NUMERO_CNPS uniques.
4. Sauvegarde en un ou plusieurs formats.
5. Log un résumé final :
    - lignes source,
    - nombre d'uniques,
    - duplications supprimées,
    - temps d'exécution.

Rôle : exécuter toute la chaîne automatiquement, du chargement à la sauvegarde.

---

7. Bloc `if __name__ == "__main__"`

Permet d'exécuter le pipeline directement :

```
df_uniques = main()
```

Rôle : fonctionnement autonome du script.

```
import pandas as pd
import os
import logging
from pathlib import Path
from typing import Optional, Dict

# ==============================================================================
# CONFIGURATION ET LOGGING
# ==============================================================================

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)
logger = logging.getLogger(__name__)

# Configuration des chemins
DOSSIER_BASE = "./CNPS_FUSION"

# ⚠️ Adapter ici le nom du fichier concaténé si besoin
#   - Si ton script de concaténation a produit un Excel :
"TRAVAILLEURS_CONCATENES_2024_2025.xlsx"
#   - Si c'était trop gros, il a peut-être produit un Parquet :
"TRAVAILLEURS_CONCATENES_2024_2025.parquet"
NOM_FICHIER_SOURCE = "TRAVAILLEURS_CONCATENES_2024_2025.parquet"

# Nom "logique" pour la sortie (base du nom, sans se soucier de l'extension)
NOM_FICHIER_SORTIE = "NUMERO_CNPS_UNIQUES.xlsx"

# ==============================================================================
# FONCTIONS I/O LOCALES
# ==============================================================================

def charger_fichier_local(nom_fichier: str, dossier: str = DOSSIER_BASE) ->
pd.DataFrame:
    """
    Charge un fichier depuis le système de fichiers local.
    Supporte automatiquement XLSX, PARQUET, CSV selon l'extension.

    Args:
        nom_fichier: Nom du fichier à charger
        dossier: Dossier contenant le fichier

    Returns:
        DataFrame chargé
```

```
    """
    try:
        chemin_complet = os.path.join(dossier, nom_fichier)
        logger.info(f"📥 Chargement : {chemin_complet}")

        if not Path(chemin_complet).exists():
            raise FileNotFoundError(f"Le fichier {chemin_complet} n'existe pas")

        ext = Path(chemin_complet).suffix.lower()

        if ext == ".xlsx":
            df = pd.read_excel(chemin_complet, engine='openpyxl')
        elif ext == ".parquet":
            df = pd.read_parquet(chemin_complet)
        elif ext == ".csv":
            df = pd.read_csv(chemin_complet, dtype=str)  # on force str si
possible
        else:
            raise ValueError(f"Extension de fichier non supportée : {ext}")

        taille_mb = Path(chemin_complet).stat().st_size / (1024 * 1024)
        logger.info(f"   ✓ {df.shape[0]:,} lignes × {df.shape[1]} colonnes
({taille_mb:.2f} MB)")

        return df

    except Exception as e:
        logger.error(f"❌ Erreur lors du chargement: {e}")
        raise


def sauvegarder_excel_local(
    df: pd.DataFrame,
    nom_fichier: str,
    dossier: str = DOSSIER_BASE
) -> str:
    """
    Sauvegarde un DataFrame en Excel dans le dossier local

    Args:
        df: DataFrame à sauvegarder
        nom_fichier: Nom du fichier de sortie
        dossier: Dossier de destination

    Returns:
        Chemin complet du fichier sauvegardé
    """
    try:
        # Créer le dossier si nécessaire
        Path(dossier).mkdir(parents=True, exist_ok=True)

        chemin_complet = os.path.join(dossier, nom_fichier)

        # Estimation de la taille
```

```
        taille_estimee_mb = df.memory_usage(deep=True).sum() / (1024 * 1024)
        logger.info(f"💾 Sauvegarde : {chemin_complet} (~{taille_estimee_mb:.2f}
MB)")

        # Sauvegarde
        df.to_excel(chemin_complet, index=False, engine='openpyxl')

        # Taille réelle
        taille_reelle_mb = Path(chemin_complet).stat().st_size / (1024 * 1024)
        logger.info(f"   ✅ Sauvegarde réussie ({taille_reelle_mb:.2f} MB)")

        return chemin_complet

    except Exception as e:
        logger.error(f"❌ Erreur lors de la sauvegarde: {e}")
        raise


# ============================================================================
# FONCTION D'EXTRACTION DES UNIQUES
# ============================================================================

def extraire_numero_cnps_uniques(
    df: pd.DataFrame,
    col_numero: str = "NUMERO_CNPS",
    avec_stats: bool = True
) -> pd.DataFrame:
    """
    Extrait les NUMERO_CNPS uniques d'un DataFrame

    Args:
        df: DataFrame source
        col_numero: Nom de la colonne contenant les NUMERO_CNPS
        avec_stats: Si True, affiche des statistiques détaillées

    Returns:
        DataFrame contenant uniquement les NUMERO_CNPS uniques
    """
    logger.info("🔍 Extraction des NUMERO_CNPS uniques...")

    # Vérification de la colonne
    if col_numero not in df.columns:
        raise KeyError(f"La colonne '{col_numero}' est absente du DataFrame")

    # On force la colonne en string pour bien gérer longueurs / analyse
    df = df.copy()
    df[col_numero] = df[col_numero].astype(str)

    # Statistiques initiales
    total = len(df)
    logger.info(f"   Total de lignes dans la source : {total:,}")

    # Extraction et dédoublonnage
    logger.info("   Suppression des doublons...")
```

```
    df_uniques = df[[col_numero]].drop_duplicates().reset_index(drop=True)
    uniques = len(df_uniques)

    # Statistiques détaillées
    if avec_stats:
        logger.info("\n📊 Statistiques :")
        logger.info(f"   Lignes totales              : {total:,}")
        logger.info(f"   NUMERO_CNPS uniques         : {uniques:,}")
        logger.info(f"   Duplications supprimées     : {total - uniques:,}")
        logger.info(f"   Taux de duplication         : {100 * (1 -
uniques/total):.2f}%")
        logger.info(f"   Taux de conservation        : {100 *
uniques/total:.2f}%")

        # Analyse de la structure des NUMERO_CNPS
        logger.info("\n🔬 Analyse de la structure :")

        # Longueur des numéros
        longueurs = df_uniques[col_numero].str.len()
        logger.info(f"   Longueurs des NUMERO_CNPS :")
        for longueur, count in longueurs.value_counts().sort_index().items():
            logger.info(f"      {longueur} caractères : {count:,} numéros
({100*count/uniques:.2f}%)")

        # Distribution par sexe (1er caractère)
        sexe_code = df_uniques[col_numero].str[0]
        logger.info(f"\n   Distribution par sexe (1er chiffre) :")
        for sexe, count in sexe_code.value_counts().sort_index().items():
            if sexe == "1":
                sexe_label = "Homme"
            elif sexe == "2":
                sexe_label = "Femme"
            else:
                sexe_label = "Inconnu"
            logger.info(f"      {sexe} ({sexe_label}) : {count:,} numéros
({100*count/uniques:.2f}%)")

    logger.info(f"\n   ✓ Extraction terminée : {uniques:,} NUMERO_CNPS uniques")

    return df_uniques


# ============================================================================
# FONCTIONS D'ANALYSE SUPPLÉMENTAIRES
# ============================================================================

def analyser_duplications(
    df: pd.DataFrame,
    col_numero: str = "NUMERO_CNPS"
) -> pd.DataFrame:
    """
    Analyse détaillée des duplications

    Args:
```

```
        df: DataFrame source
        col_numero: Nom de la colonne NUMERO_CNPS

    Returns:
        DataFrame avec les statistiques de duplication
    """
    logger.info("\n📊 Analyse détaillée des duplications...")

    # On force en string pour être cohérent
    df = df.copy()
    df[col_numero] = df[col_numero].astype(str)

    # Compter les occurrences de chaque NUMERO_CNPS
    comptage = df[col_numero].value_counts()

    # Statistiques
    logger.info(f"   Total de NUMERO_CNPS différents : {len(comptage):,}")
    logger.info(f"   NUMERO_CNPS apparaissant 1 fois : {(comptage == 1).sum():,}")
    logger.info(f"   NUMERO_CNPS dupliqués           : {(comptage > 1).sum():,}")

    if (comptage > 1).any():
        max_duplication = comptage.max()
        logger.info(f"   Duplication maximale            : {max_duplication}
fois")

        # Distribution des duplications
        logger.info("\n   Distribution des duplications :")
        dist_duplication = comptage[comptage > 1].value_counts().sort_index()
        for nb_occur, nb_numeros in dist_duplication.head(10).items():
            logger.info(f"      {nb_occur} occurrences : {nb_numeros:,} numéros")

        # Top 10 des numéros les plus dupliqués
        logger.info("\n   Top 10 des NUMERO_CNPS les plus fréquents :")
        for i, (numero, count) in enumerate(comptage.head(10).items(), 1):
            logger.info(f"      {i:2d}. {numero} : {count} occurrences")

        # Créer un DataFrame récapitulatif
        df_stats = pd.DataFrame({
            'NUMERO_CNPS': comptage.index,
            'Nombre_Occurrences': comptage.values
        })

        return df_stats
    else:
        logger.info("   ✓ Aucune duplication détectée")
        return pd.DataFrame()


def sauvegarder_formats_multiples(
    df: pd.DataFrame,
    nom_base: str = "NUMERO_CNPS_UNIQUES",
    dossier: str = DOSSIER_BASE
) -> Dict[str, str]:
    """
```

```
    Sauvegarde le DataFrame dans plusieurs formats

    Args:
        df: DataFrame à sauvegarder
        nom_base: Nom de base du fichier (sans extension)
        dossier: Dossier de destination

    Returns:
        Dictionnaire avec les chemins des fichiers créés
    """
    logger.info("\n💾 Sauvegarde dans plusieurs formats...")

    fichiers = {}

    # Excel
    fichier_xlsx = sauvegarder_excel_local(df, f"{nom_base}.xlsx", dossier)
    fichiers['xlsx'] = fichier_xlsx

    # CSV (plus léger)
    fichier_csv = os.path.join(dossier, f"{nom_base}.csv")
    df.to_csv(fichier_csv, index=False, encoding='utf-8-sig')
    taille_csv = Path(fichier_csv).stat().st_size / (1024 * 1024)
    logger.info(f"   ✓ CSV sauvegardé : {fichier_csv} ({taille_csv:.2f} MB)")
    fichiers['csv'] = fichier_csv

    # TXT (liste simple)
    fichier_txt = os.path.join(dossier, f"{nom_base}.txt")
    df['NUMERO_CNPS'].to_csv(fichier_txt, index=False, header=False)
    taille_txt = Path(fichier_txt).stat().st_size / (1024 * 1024)
    logger.info(f"   ✓ TXT sauvegardé : {fichier_txt} ({taille_txt:.2f} MB)")
    fichiers['txt'] = fichier_txt

    return fichiers


# ============================================================================
# FONCTION PRINCIPALE
# ============================================================================

def main(
    nom_fichier_source: str = NOM_FICHIER_SOURCE,
    nom_fichier_sortie: str = NOM_FICHIER_SORTIE,
    analyse_duplications: bool = True,
    multi_formats: bool = True
) -> Optional[pd.DataFrame]:
    """
    Pipeline principal d'extraction des NUMERO_CNPS uniques

    Args:
        nom_fichier_source: Nom du fichier source (xlsx / parquet / csv)
        nom_fichier_sortie: Nom du fichier de sortie (pour Excel / base du nom)
        analyse_duplications: Si True, effectue une analyse détaillée des
duplications
        multi_formats: Si True, sauvegarde dans plusieurs formats
```

```
    Returns:
        DataFrame contenant les NUMERO_CNPS uniques
    """
    import time
    start_time = time.time()

    logger.info("=" * 80)
    logger.info("🚀 EXTRACTION DES NUMERO_CNPS UNIQUES (VERSION LOCALE)")
    logger.info("=" * 80)

    try:
        # ÉTAPE 1: Chargement du fichier source
        logger.info("\n📥 ÉTAPE 1 : CHARGEMENT DU FICHIER SOURCE")
        logger.info("-" * 80)
        df = charger_fichier_local(nom_fichier_source)

        # ÉTAPE 2: Analyse des duplications (optionnel)
        if analyse_duplications:
            logger.info("\n📊 ÉTAPE 2 : ANALYSE DES DUPLICATIONS")
            logger.info("-" * 80)
            df_stats_duplication = analyser_duplications(df)

            # Sauvegarder les statistiques si des duplications existent
            if not df_stats_duplication.empty:
                chemin_stats = os.path.join(DOSSIER_BASE,
"STATS_DUPLICATIONS_NUMERO_CNPS.xlsx")
                df_stats_duplication.to_excel(chemin_stats, index=False)
                logger.info(f"   💾 Statistiques de duplication sauvegardées :
{chemin_stats}")

        # ÉTAPE 3: Extraction des uniques
        logger.info("\n🔍 ÉTAPE 3 : EXTRACTION DES NUMERO_CNPS UNIQUES")
        logger.info("-" * 80)
        df_uniques = extraire_numero_cnps_uniques(df, avec_stats=True)

        # ÉTAPE 4: Sauvegarde
        logger.info("\n💾 ÉTAPE 4 : SAUVEGARDE")
        logger.info("-" * 80)

        if multi_formats:
            base_nom = nom_fichier_sortie.replace('.xlsx', '')
            fichiers = sauvegarder_formats_multiples(
                df_uniques,
                nom_base=base_nom,
                dossier=DOSSIER_BASE
            )
            logger.info("\n   📂 Fichiers créés :")
            for format_type, chemin in fichiers.items():
                logger.info(f"      - {format_type.upper():5s} : {chemin}")
        else:
            chemin_sortie = sauvegarder_excel_local(df_uniques,
nom_fichier_sortie)
            logger.info(f"   📍 Fichier disponible : {chemin_sortie}")
```

```
        # Résumé final
        elapsed_time = time.time() - start_time
        logger.info("\n" + "=" * 80)
        logger.info("✅ EXTRACTION TERMINÉE AVEC SUCCÈS")
        logger.info(f"⏱️  Temps d'exécution : {elapsed_time:.2f} secondes")
        logger.info(f"📊 Résultat :")
        logger.info(f"   - Fichier source         : {nom_fichier_source}")
        logger.info(f"   - Lignes source          : {len(df):,}")
        logger.info(f"   - NUMERO_CNPS uniques    : {len(df_uniques):,}")
        logger.info(f"   - Duplications supprimées: {len(df) -
len(df_uniques):,}")
        logger.info(f"   - Localisation           : {DOSSIER_BASE}/")
        logger.info("=" * 80)

        return df_uniques

    except FileNotFoundError as e:
        logger.error(f"\n❌ FICHIER INTROUVABLE: {e}")
        logger.info("💡 Assurez-vous d'avoir exécuté le script de concaténation
d'abord")
        return None

    except Exception as e:
        logger.error(f"\n❌ ERREUR CRITIQUE: {e}", exc_info=True)
        raise


# ============================================================================
# POINT D'ENTRÉE
# ============================================================================

if __name__ == "__main__":
    df_uniques = main()
```

```
2025-12-04 15:31:30 - INFO -
================================================================================
2025-12-04 15:31:30 - INFO - 🚀 EXTRACTION DES NUMERO_CNPS UNIQUES (VERSION
LOCALE)
2025-12-04 15:31:30 - INFO -
================================================================================
2025-12-04 15:31:30 - INFO -
📥 ÉTAPE 1 : CHARGEMENT DU FICHIER SOURCE
2025-12-04 15:31:30 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 15:31:30 - INFO - 📥 Chargement :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025.parquet
2025-12-04 15:31:31 - INFO -    ✓ 1,503,000 lignes × 14 colonnes (42.99 MB)
2025-12-04 15:31:31 - INFO -
📊 ÉTAPE 2 : ANALYSE DES DUPLICATIONS
```

```
2025-12-04 15:31:31 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 15:31:31 - INFO -
📊 Analyse détaillée des duplications...
2025-12-04 15:31:32 - INFO -    Total de NUMERO_CNPS différents : 502,000
2025-12-04 15:31:32 - INFO -    NUMERO_CNPS apparaissant 1 fois : 1,000
2025-12-04 15:31:32 - INFO -    NUMERO_CNPS dupliqués           : 501,000
2025-12-04 15:31:32 - INFO -    Duplication maximale            : 3 fois
2025-12-04 15:31:32 - INFO -
   Distribution des duplications :
2025-12-04 15:31:32 - INFO -       2 occurrences : 1,000 numéros
2025-12-04 15:31:32 - INFO -       3 occurrences : 500,000 numéros
2025-12-04 15:31:32 - INFO -
   Top 10 des NUMERO_CNPS les plus fréquents :
2025-12-04 15:31:32 - INFO -        1. 175070409808 : 3 occurrences
2025-12-04 15:31:32 - INFO -        2. 190060006564 : 3 occurrences
2025-12-04 15:31:32 - INFO -        3. 196020222089 : 3 occurrences
2025-12-04 15:31:32 - INFO -        4. 179090307526 : 3 occurrences
2025-12-04 15:31:32 - INFO -        5. 100010133300 : 3 occurrences
2025-12-04 15:31:32 - INFO -        6. 200010492573 : 3 occurrences
2025-12-04 15:31:32 - INFO -        7. 186100218818 : 3 occurrences
2025-12-04 15:31:32 - INFO -        8. 292070178645 : 3 occurrences
2025-12-04 15:31:32 - INFO -        9. 100010488582 : 3 occurrences
2025-12-04 15:31:32 - INFO -       10. 195120118834 : 3 occurrences
2025-12-04 15:31:51 - INFO -    💾 Statistiques de duplication sauvegardées :
./CNPS_FUSION/STATS_DUPLICATIONS_NUMERO_CNPS.xlsx
2025-12-04 15:31:51 - INFO -
🔍 ÉTAPE 3 : EXTRACTION DES NUMERO_CNPS UNIQUES
2025-12-04 15:31:51 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 15:31:51 - INFO - 🔍 Extraction des NUMERO_CNPS uniques...
2025-12-04 15:31:51 - INFO -    Total de lignes dans la source : 1,503,000
2025-12-04 15:31:51 - INFO -    Suppression des doublons...
2025-12-04 15:31:51 - INFO -
📊 Statistiques :
2025-12-04 15:31:51 - INFO -    Lignes totales              : 1,503,000
2025-12-04 15:31:51 - INFO -    NUMERO_CNPS uniques         : 502,000
2025-12-04 15:31:51 - INFO -    Duplications supprimées     : 1,001,000
2025-12-04 15:31:51 - INFO -    Taux de duplication         : 66.60%
2025-12-04 15:31:51 - INFO -    Taux de conservation        : 33.40%
2025-12-04 15:31:51 - INFO -
🔬 Analyse de la structure :
2025-12-04 15:31:52 - INFO -    Longueurs des NUMERO_CNPS :
2025-12-04 15:31:52 - INFO -       12 caractères : 502,000 numéros (100.00%)
2025-12-04 15:31:52 - INFO -
   Distribution par sexe (1er chiffre) :
2025-12-04 15:31:52 - INFO -       1 (Homme) : 385,469 numéros (76.79%)
2025-12-04 15:31:52 - INFO -       2 (Femme) : 116,531 numéros (23.21%)
2025-12-04 15:31:52 - INFO -
   ✓ Extraction terminée : 502,000 NUMERO_CNPS uniques
2025-12-04 15:31:52 - INFO -
```

```
💾 ÉTAPE 4 : SAUVEGARDE
2025-12-04 15:31:52 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 15:31:52 - INFO -
💾 Sauvegarde dans plusieurs formats...
2025-12-04 15:31:52 - INFO - 💾 Sauvegarde :
./CNPS_FUSION/NUMERO_CNPS_UNIQUES.xlsx (~29.20 MB)
2025-12-04 15:32:07 - INFO -    ☑ Sauvegarde réussie (6.01 MB)
2025-12-04 15:32:07 - INFO -    ✓ CSV sauvegardé :
./CNPS_FUSION/NUMERO_CNPS_UNIQUES.csv (6.22 MB)
2025-12-04 15:32:08 - INFO -    ✓ TXT sauvegardé :
./CNPS_FUSION/NUMERO_CNPS_UNIQUES.txt (6.22 MB)
2025-12-04 15:32:08 - INFO -
   🗁 Fichiers créés :
2025-12-04 15:32:08 - INFO -       - XLSX  :
./CNPS_FUSION/NUMERO_CNPS_UNIQUES.xlsx
2025-12-04 15:32:08 - INFO -       - CSV   : ./CNPS_FUSION/NUMERO_CNPS_UNIQUES.csv
2025-12-04 15:32:08 - INFO -       - TXT   : ./CNPS_FUSION/NUMERO_CNPS_UNIQUES.txt
2025-12-04 15:32:08 - INFO -
================================================================================
2025-12-04 15:32:08 - INFO - ☑ EXTRACTION TERMINÉE AVEC SUCCÈS
2025-12-04 15:32:08 - INFO - ⏱  Temps d'exécution : 37.93 secondes
2025-12-04 15:32:08 - INFO - 📊 Résultat :
2025-12-04 15:32:08 - INFO -    - Fichier source         :
TRAVAILLEURS_CONCATENES_2024_2025.parquet
2025-12-04 15:32:08 - INFO -    - Lignes source          : 1,503,000
2025-12-04 15:32:08 - INFO -    - NUMERO_CNPS uniques    : 502,000
2025-12-04 15:32:08 - INFO -    - Duplications supprimées: 1,001,000
2025-12-04 15:32:08 - INFO -    - Localisation           : ./CNPS_FUSION/
2025-12-04 15:32:08 - INFO -
================================================================================
```

Etape 3: Appliquer le hachage HMAC sur les matricules uniques

🎯 Objectif

Ce script prend la base `NUMERO_CNPS_UNIQUES` (CNPS déjà dédoublonnés) et génère pour chaque numéro un **ID_ANSTAT pseudonymisé** en utilisant un **Algorithme de Hachage crypto"** :

- HMAC-SHA256 (ou PBKDF2-HMAC-SHA256),
- gestion sérieuse d'une **clé secrète**,
- sauvegarde du résultat (NUMERO_CNPS + ID_ANSTAT) en **Excel + CSV**.

🧩 Les différentes étapes

---

1. Gestion de la clé secrète (le “sel du patron”)

`generer_sel_unique()`

- Génère une clé/sel cryptographique de 32 octets (64 caractères hex).
- À utiliser une fois pour créer la **clé maîtresse**.
- Cette clé doit ensuite être stockée dans un endroit sécurisé (vault, fichier protégé, variable d’environnement).

`charger_cle_secrete_depuis_env(varname="ANSTAT_SECRET_KEY", allow_generate=False)`

- Tente de lire la clé depuis la variable d’environnement `ANSTAT_SECRET_KEY`.
- Si absente :
  - lève une erreur (par défaut),
  - ou génère une **clé temporaire** si `allow_generate=True` (usage test/local uniquement).

`sauvegarder_cle_secrete(cle, fichier="./cle_secrete_anstat.txt")`

- Écrit la clé dans un fichier texte.
- Applique des permissions Unix restrictives `0o400` (lecture seule pour le propriétaire).
- Sert pour garder une copie locale sécurisée de la clé.

`charger_cle_secrete_depuis_fichier(fichier="./cle_secrete_anstat.txt")`

- Lit la clé depuis un fichier déjà créé.
- Utilisé dans `main()` si le fichier existe et qu’on ne demande pas de nouvelle clé.

---

2. Génération des ID_ANSTAT (pseudonymisation)

`generer_id_anstat(numero_cnps, cle_secrete)`

- Fonction principale de pseudonymisation.
- Utilise **HMAC-SHA256** avec :
  - message = `numero_cnps` nettoyé (strip),
  - clé = `cle_secrete`.
- Retourne un **hash hexadécimal de 64 caractères**.

Propriétés :

- Déterministe : même CNPS + même clé → même ID_ANSTAT.
- Unidirectionnel : sans la clé, on ne revient pas aux CNPS.
- Collisions très peu probables.

`generer_id_anstat_pbkdf2(numero_cnps, cle_secrete, iterations=100_000)`

- Variante plus coûteuse mais plus résistante aux attaques par dictionnaire.
- Utilise PBKDF2-HMAC-SHA256 avec `iterations` (par défaut 100 000).
- Retourne aussi un hex de 64 caractères.

Usage : option "blindage maximal" si on craint des attaques par force brute sur un petit espace de valeurs.

---

3. Lecture / écriture des fichiers

`charger_fichier_numero_cnps(format_preference="xlsx", dossier=DOSSIER_BASE)`

- Charge le fichier `NUMERO_CNPS_UNIQUES` :
  - soit en **Excel** (`NUMERO_CNPS_UNIQUES.xlsx`),
  - soit en **CSV** (`NUMERO_CNPS_UNIQUES.csv`) selon `format_preference`.
- Vérifie que la colonne `NUMERO_CNPS` existe.
- Log le nombre de lignes chargées.

`sauvegarder_resultat(df, nom_fichier=FICHIER_SORTIE, dossier=DOSSIER_BASE)`

- Sauvegarde le DataFrame (avec `NUMERO_CNPS` + `ID_ANSTAT`) en deux formats :
  - Excel : `NUMERO_CNPS_UNIQUES_AVEC_ID_ANSTAT.xlsx`,
  - CSV : `NUMERO_CNPS_UNIQUES_AVEC_ID_ANSTAT.csv`.
- Log la taille de chaque fichier.

---

4. Pipeline principal `main(...)`

Paramètres importants :

- `utiliser_pbkdf2` : choisir entre HMAC-SHA256 (False) ou PBKDF2 (True),
- `generer_nouvelle_cle` : générer une clé si aucune clé n'existe,
- `sauvegarder_cle` : sauver la clé générée dans un fichier,
- `format_source` : `xlsx` ou `csv`.

Étapes du pipeline

1. **Chargement des NUMERO_CNPS** → via `charger_fichier_numero_cnps`.
2. **Chargement de la clé secrète**

- Si `fichier_cle` existe et `generer_nouvelle_cle=False` → on lit la clé depuis le fichier.
- Sinon → on tente la variable d'environnement `ANSTAT_SECRET_KEY`, ou on génère une clé temporaire si autorisé.

3. **Génération des ID_ANSTAT**
   - Application de `generer_id_anstat` ou `generer_id_anstat_pbkdf2` sur chaque `NUMERO_CNPS`.
   - Contrôles :
     - nombre d'ID_ANSTAT uniques,
     - détection éventuelle de collisions (très improbable).
4. **Sauvegarde des résultats**
   - Appel à `sauvegarder_resultat` (Excel + CSV).
   - Logs récapitulatifs (nombre de lignes, méthode utilisée, temps d'exécution).
5. **Message de sécurité**
   - Si une clé temporaire a été générée, rappel : **ne jamais utiliser ça en production**.

```python
import os
import secrets
import hashlib
import hmac
import pandas as pd
import logging
from pathlib import Path
from typing import Optional
from datetime import datetime

# ============================================================================
# CONFIGURATION ET LOGGING
# ============================================================================

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)
logger = logging.getLogger(__name__)

# Configuration des chemins
DOSSIER_BASE = "./CNPS_FUSION"
FICHIER_SOURCE_XLSX = "NUMERO_CNPS_UNIQUES.xlsx"
FICHIER_SOURCE_CSV = "NUMERO_CNPS_UNIQUES.csv"
FICHIER_SORTIE = "NUMERO_CNPS_UNIQUES_AVEC_ID_ANSTAT.xlsx"

# ============================================================================
# 1. GESTION DU SEL SECRET (CODE DU PATRON)
```

```python
# ==============================================================================

def generer_sel_unique() -> str:
    """
    Génère un sel cryptographiquement sûr de 32 octets (64 caractères
hexadécimaux).

    Cette fonction peut servir lors de l'installation initiale pour créer une clé
secrète
    qui devra ensuite être stockée de manière sécurisée (vault, gestionnaire de
secrets, env var chiffrée...).

    NOTE: En production, NE PAS générer la clé automatiquement à l'import —
chargez-la depuis
    un gestionnaire de secrets persistant. Cette fonction est disponible pour
générer
    une clé lors de l'initialisation.

    Returns:
        str: Clé secrète de 64 caractères hexadécimaux
    """
    return secrets.token_hex(32)


def charger_cle_secrete_depuis_env(
    varname: str = "ANSTAT_SECRET_KEY",
    allow_generate: bool = False
) -> str:
    """
    Charge la clé secrète depuis une variable d'environnement.

    - Si la variable existe, elle est renvoyée.
    - Si elle n'existe pas et allow_generate=True, la fonction génère une clé
temporaire
      (utile pour tests ou usages locaux).
    - Si elle n'existe pas et allow_generate=False, lève une erreur pour forcer
      l'utilisation d'une clé persistante et sûre.

    Args:
        varname: Nom de la variable d'environnement
        allow_generate: Si True, génère une clé temporaire si absente

    Returns:
        str: Clé secrète

    Raises:
        EnvironmentError: Si la clé n'est pas définie et allow_generate=False
    """
    val = os.getenv(varname)
    if val:
        logger.info(f"✓ Clé secrète chargée depuis la variable '{varname}'")
        return val

    if allow_generate:
```

```
        logger.warning("⚠ ATTENTION : Variable ANSTAT_SECRET_KEY absente.")
        logger.warning("   Génération d'une clé TEMPORAIRE pour tests
uniquement.")
        logger.warning("   NE PAS UTILISER EN PRODUCTION !")
        cle_temp = generer_sel_unique()
        logger.info(f"   Clé temporaire générée : {cle_temp[:16]}... (tronquée)")
        return cle_temp

    raise EnvironmentError(
        f"La variable d'environnement '{varname}' n'est pas définie. "
        "Configurez la clé secrète de pseudonymisation de façon sécurisée."
    )


def sauvegarder_cle_secrete(cle: str, fichier: str = "./cle_secrete_anstat.txt") -
> None:
    """
    Sauvegarde la clé secrète dans un fichier protégé

    ⚠ IMPORTANT : Ce fichier doit être protégé et ne jamais être partagé

    Args:
        cle: Clé secrète à sauvegarder
        fichier: Chemin du fichier de destination
    """
    logger.warning("⚠ Sauvegarde de la clé secrète dans un fichier")
    logger.warning(f"   Fichier : {fichier}")
    logger.warning("   PROTÉGEZ ce fichier et NE LE PARTAGEZ JAMAIS")

    with open(fichier, 'w') as f:
        f.write(cle)

    # Définir les permissions (lecture seule pour le propriétaire)
    os.chmod(fichier, 0o400)

    logger.info(f"   ✓ Clé sauvegardée avec permissions restrictives (400)")


def charger_cle_secrete_depuis_fichier(fichier: str = "./cle_secrete_anstat.txt")
-> str:
    """
    Charge la clé secrète depuis un fichier

    Args:
        fichier: Chemin du fichier contenant la clé

    Returns:
        Clé secrète
    """
    if not Path(fichier).exists():
        raise FileNotFoundError(f"Fichier de clé introuvable : {fichier}")

    with open(fichier, 'r') as f:
        cle = f.read().strip()
```

```
    logger.info(f"✓ Clé secrète chargée depuis : {fichier}")
    return cle


# ==============================================================================
# 2. FONCTION DE HACHAGE SALÉ (CODE DU PATRON)
# ==============================================================================

def generer_id_anstat(numero_cnps: str, cle_secrete: str) -> str:
    """
    Crée un pseudonyme de façon sûre en utilisant HMAC-SHA256.

    Cette version utilise HMAC plutôt que la simple concaténation + SHA256.
    HMAC évite certaines erreurs de concaténation et permet d'utiliser une
    clé secrète comme véritable clé MAC.

    Arguments:
        numero_cnps (str): Le numéro CNPS à anonymiser.
        cle_secrete (str): La clé secrète persistante (format hexadécimal ou
chaîne).

    Retourne:
        str: Le pseudonyme HMAC-SHA256 en hexadécimal (64 caractères).
    """
    if not isinstance(numero_cnps, (str, bytes)):
        raise TypeError(f"numero_cnps doit être une chaîne ou bytes, pas
{type(numero_cnps)}")

    # Canonicalisation minimale : retirer espaces et normaliser
    numero = numero_cnps.strip()

    key_bytes = cle_secrete.encode("utf-8") if isinstance(cle_secrete, str) else
cle_secrete
    msg = numero.encode("utf-8") if isinstance(numero, str) else numero

    mac = hmac.new(key_bytes, msg, digestmod=hashlib.sha256)
    return mac.hexdigest()


def generer_id_anstat_pbkdf2(
    numero_cnps: str,
    cle_secrete: str,
    iterations: int = 100_000
) -> str:
    """
    Alternative résistante aux attaques par force brute : PBKDF2-HMAC-SHA256.

    Note: PBKDF2 rendra la dérivation du pseudonyme plus coûteuse (utile si le
    numéro est dans un petit espace et sujet aux attaques par dictionnaire).
    L'output sera en hexadécimal.

    Args:
        numero_cnps: Numéro CNPS à hacher
```

```
        cle_secrete: Clé secrète
        iterations: Nombre d'itérations (défaut 100 000)

    Returns:
        str: Pseudonyme PBKDF2 (64 caractères hexadécimaux)
    """
    if not isinstance(numero_cnps, (str, bytes)):
        raise TypeError(f"numero_cnps doit être une chaîne ou bytes, pas
{type(numero_cnps)}")

    numero = numero_cnps.strip()
    key_bytes = cle_secrete.encode("utf-8") if isinstance(cle_secrete, str) else
cle_secrete
    derived = hashlib.pbkdf2_hmac('sha256', numero.encode('utf-8'), key_bytes,
iterations)
    return derived.hex()


# ============================================================================
# 3. FONCTIONS I/O
# ============================================================================

def charger_fichier_numero_cnps(
    format_preference: str = "xlsx",
    dossier: str = DOSSIER_BASE
) -> pd.DataFrame:
    """
    Charge le fichier des NUMERO_CNPS uniques

    Args:
        format_preference: "xlsx" ou "csv"
        dossier: Dossier contenant le fichier

    Returns:
        DataFrame des NUMERO_CNPS
    """
    if format_preference.lower() == "csv":
        fichier = os.path.join(dossier, FICHIER_SOURCE_CSV)
        logger.info(f"📥 Chargement : {fichier}")
        df = pd.read_csv(fichier)
    else:
        fichier = os.path.join(dossier, FICHIER_SOURCE_XLSX)
        logger.info(f"📥 Chargement : {fichier}")
        df = pd.read_excel(fichier, engine='openpyxl')

    logger.info(f"   ✓ {len(df):,} NUMERO_CNPS chargés")

    if "NUMERO_CNPS" not in df.columns:
        raise KeyError(
            f"La colonne 'NUMERO_CNPS' est absente du fichier. "
            f"Colonnes disponibles : {list(df.columns)}"
        )

    return df
```

```python
def sauvegarder_resultat(
    df: pd.DataFrame,
    nom_fichier: str = FICHIER_SORTIE,
    dossier: str = DOSSIER_BASE
) -> dict:
    """
    Sauvegarde le résultat dans plusieurs formats

    Args:
        df: DataFrame avec NUMERO_CNPS et ID_ANSTAT
        nom_fichier: Nom du fichier de base
        dossier: Dossier de destination

    Returns:
        dict: Chemins des fichiers créés
    """
    Path(dossier).mkdir(parents=True, exist_ok=True)
    fichiers = {}

    nom_base = nom_fichier.replace('.xlsx', '')

    # Excel
    chemin_xlsx = os.path.join(dossier, f"{nom_base}.xlsx")
    logger.info(f"💾 Sauvegarde EXCEL : {chemin_xlsx}")
    df.to_excel(chemin_xlsx, index=False, engine='openpyxl')
    taille_xlsx = Path(chemin_xlsx).stat().st_size / (1024 * 1024)
    logger.info(f"   ✅ Excel sauvegardé : {taille_xlsx:.2f} MB")
    fichiers['xlsx'] = chemin_xlsx

    # CSV
    chemin_csv = os.path.join(dossier, f"{nom_base}.csv")
    logger.info(f"💾 Sauvegarde CSV : {chemin_csv}")
    df.to_csv(chemin_csv, index=False, encoding='utf-8-sig')
    taille_csv = Path(chemin_csv).stat().st_size / (1024 * 1024)
    logger.info(f"   ✅ CSV sauvegardé : {taille_csv:.2f} MB")
    fichiers['csv'] = chemin_csv

    return fichiers


# ============================================================================
# 4. FONCTION PRINCIPALE
# ============================================================================

def main(
    utiliser_pbkdf2: bool = False,
    iterations_pbkdf2: int = 100_000,
    generer_nouvelle_cle: bool = False,
    sauvegarder_cle: bool = False,
    fichier_cle: str = "./cle_secrete_anstat.txt",
    format_source: str = "xlsx"
) -> Optional[pd.DataFrame]:
```

```
    """
    Pipeline principal de génération des ID_ANSTAT

    Args:
        utiliser_pbkdf2: Si True, utilise PBKDF2 au lieu de HMAC simple
        iterations_pbkdf2: Nombre d'itérations pour PBKDF2
        generer_nouvelle_cle: Si True, génère une clé temporaire
        sauvegarder_cle: Si True, sauvegarde la clé générée
        fichier_cle: Chemin du fichier de clé
        format_source: "xlsx" ou "csv"

    Returns:
        DataFrame avec NUMERO_CNPS et ID_ANSTAT
    """
    import time
    start_time = time.time()

    logger.info("=" * 80)
    logger.info("🚀 GÉNÉRATION ID_ANSTAT (CODE DU PATRON - HMAC-SHA256)")
    logger.info("=" * 80)

    try:
        # ÉTAPE 1: Chargement du fichier source
        logger.info("\n📥 ÉTAPE 1 : CHARGEMENT DU FICHIER SOURCE")
        logger.info("-" * 80)
        df = charger_fichier_numero_cnps(format_preference=format_source)

        # ÉTAPE 2: Chargement de la clé secrète
        logger.info("\n🔐 ÉTAPE 2 : CHARGEMENT DE LA CLÉ SECRÈTE")
        logger.info("-" * 80)

        # Essayer de charger depuis un fichier d'abord
        if Path(fichier_cle).exists() and not generer_nouvelle_cle:
            logger.info(f"   Mode : Chargement depuis fichier existant")
            cle_secrete = charger_cle_secrete_depuis_fichier(fichier_cle)
        else:
            # Sinon, charger depuis variable d'environnement ou générer
            logger.info(f"   Mode : Variable d'environnement / Génération")
            cle_secrete = charger_cle_secrete_depuis_env(
                varname="ANSTAT_SECRET_KEY",
                allow_generate=generer_nouvelle_cle
            )

            # Sauvegarder la clé si demandé
            if generer_nouvelle_cle and sauvegarder_cle:
                sauvegarder_cle_secrete(cle_secrete, fichier_cle)

        logger.info(f"   ✓ Clé secrète chargée (longueur: {len(cle_secrete)}
caractères)")

        # ÉTAPE 3: Génération des ID_ANSTAT
        logger.info("\n🔐 ÉTAPE 3 : GÉNÉRATION DES ID_ANSTAT")
        logger.info("-" * 80)
```

```
        if utiliser_pbkdf2:
            logger.info(f"   Méthode : PBKDF2-HMAC-SHA256 ({iterations_pbkdf2:,} itérations)")
            logger.info("   ⏱  ATTENTION : Plus lent mais plus sécurisé contre les attaques")

            df["ID_ANSTAT"] = df["NUMERO_CNPS"].astype(str).apply(
                lambda x: generer_id_anstat_pbkdf2(x, cle_secrete, iterations_pbkdf2)
            )
        else:
            logger.info("   Méthode : HMAC-SHA256")
            logger.info("   Propriétés :")
            logger.info("     - Déterministe (même entrée → même sortie)")
            logger.info("     - Unidirectionnel (impossible de retrouver le NUMERO_CNPS)")
            logger.info("     - Sécurisé (nécessite la clé secrète)")

            df["ID_ANSTAT"] = df["NUMERO_CNPS"].astype(str).apply(
                lambda x: generer_id_anstat(x, cle_secrete)
            )

        logger.info(f"   ✓ {len(df):,} ID_ANSTAT générés")

        # Vérifications
        logger.info("\n   Vérifications :")
        n_uniques = df["ID_ANSTAT"].nunique()
        logger.info(f"     - ID_ANSTAT uniques : {n_uniques:,}")

        if n_uniques != len(df):
            logger.warning(f"     ⚠ {len(df) - n_uniques} collisions détectées")
        else:
            logger.info(f"     ✅ Tous les ID_ANSTAT sont uniques")

        # Exemples
        logger.info("\n   Exemples (5 premiers) :")
        for idx, row in df.head(5).iterrows():
            logger.info(f"     {row['NUMERO_CNPS']} → {row['ID_ANSTAT']}")

        # ÉTAPE 4: Sauvegarde
        logger.info("\n💾 ÉTAPE 4 : SAUVEGARDE")
        logger.info("-" * 80)
        fichiers = sauvegarder_resultat(df)

        logger.info(f"\n   📂 Fichiers créés :")
        for format_type, chemin in fichiers.items():
            logger.info(f"     - {format_type.upper():5s} : {chemin}")

        # Résumé final
        elapsed_time = time.time() - start_time
        logger.info("\n" + "=" * 80)
        logger.info("✅ GÉNÉRATION ID_ANSTAT TERMINÉE AVEC SUCCÈS")
        logger.info(f"⏱  Temps d'exécution : {elapsed_time:.2f} secondes")
        logger.info(f"📊 Résultat :")
```

```
        logger.info(f"   - NUMERO_CNPS traités  : {len(df):,}")
        logger.info(f"   - ID_ANSTAT générés    : {len(df):,}")
        logger.info(f"   - ID_ANSTAT uniques    : {df['ID_ANSTAT'].nunique():,}")
        logger.info(f"   - Méthode              : {'PBKDF2' if utiliser_pbkdf2
else 'HMAC-SHA256'}")
        logger.info("=" * 80)

        # Avertissement sécurité
        if generer_nouvelle_cle:
            logger.warning("\n⚠️ RAPPEL SÉCURITÉ :")
            logger.warning("   Une clé temporaire a été générée pour ce test.")
            logger.warning("   En production, utilisez une clé secrète fixe et
sécurisée.")
            if sauvegarder_cle:
                logger.warning(f"   Clé sauvegardée dans : {fichier_cle}")
                logger.warning("   PROTÉGEZ ce fichier et ne le partagez JAMAIS.")

        return df

    except FileNotFoundError as e:
        logger.error(f"\n❌ FICHIER INTROUVABLE: {e}")
        logger.info("💡 Assurez-vous d'avoir exécuté le script d'extraction des
uniques")
        return None

    except Exception as e:
        logger.error(f"\n❌ ERREUR CRITIQUE: {e}", exc_info=True)
        raise


# ============================================================================
# 5. EXEMPLE D'UTILISATION (CODE DU PATRON ADAPTÉ)
# ============================================================================

def exemple_utilisation():
    """
    Exemple d'utilisation pour tests locaux uniquement.

    En production, la clé secrète DOIT être chargée depuis un coffre-fort sécurisé
    et ne JAMAIS être affichée.
    """
    logger.info("\n" + "=" * 80)
    logger.info("🔑 EXEMPLE D'UTILISATION (TESTS UNIQUEMENT)")
    logger.info("=" * 80)

    # Charger ou générer une clé
    key = charger_cle_secrete_depuis_env('ANSTAT_SECRET_KEY', allow_generate=True)

    cnps_exemple_1 = "194011724471"
    cnps_exemple_2 = "194011724472"

    id_anstat_1 = generer_id_anstat(cnps_exemple_1, key)
    id_anstat_2 = generer_id_anstat(cnps_exemple_2, key)
```

```
    logger.info("\n--- RÉSULTATS (Exemple) ---")
    logger.info(f"Numéro CNPS 1 : {cnps_exemple_1}")
    logger.info(f"ID ANSTAT 1 (Pseudonyme) : {id_anstat_1}")

    logger.info(f"\nNuméro CNPS 2 : {cnps_exemple_2}")
    logger.info(f"ID ANSTAT 2 (Pseudonyme) : {id_anstat_2}")
    logger.info("\nNote: Même une petite différence dans le CNPS produit un ID
complètement différent (robustesse).")


# ============================================================================
# POINT D'ENTRÉE
# ============================================================================

if __name__ == "__main__":
    # Configuration par défaut : génère une clé temporaire et la sauvegarde
    df_ids = main(
        utiliser_pbkdf2=False,          # HMAC-SHA256 standard (plus rapide)
        generer_nouvelle_cle=True,      # Génère une clé temporaire pour test
        sauvegarder_cle=True,           # Sauvegarde pour réutilisation
        format_source="xlsx"            # Utilise le fichier Excel
    )

    # Décommenter pour voir l'exemple
    # exemple_utilisation()
```

```
2025-12-04 15:43:32 - INFO -
================================================================================
2025-12-04 15:43:32 - INFO - 🚀 GÉNÉRATION ID_ANSTAT (CODE DU PATRON - HMAC-
SHA256)
2025-12-04 15:43:32 - INFO -
================================================================================
2025-12-04 15:43:32 - INFO -
📥 ÉTAPE 1 : CHARGEMENT DU FICHIER SOURCE
2025-12-04 15:43:32 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 15:43:32 - INFO - 📥 Chargement :
./CNPS_FUSION/NUMERO_CNPS_UNIQUES.xlsx
2025-12-04 15:43:46 - INFO -    ✓ 502,000 NUMERO_CNPS chargés
2025-12-04 15:43:46 - INFO -
🔐 ÉTAPE 2 : CHARGEMENT DE LA CLÉ SECRÈTE
2025-12-04 15:43:46 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 15:43:46 - INFO -    Mode : Variable d'environnement / Génération
2025-12-04 15:43:46 - WARNING - ⚠ ATTENTION : Variable ANSTAT_SECRET_KEY
absente.
2025-12-04 15:43:46 - WARNING -    Génération d'une clé TEMPORAIRE pour tests
uniquement.
2025-12-04 15:43:46 - WARNING -    NE PAS UTILISER EN PRODUCTION !
2025-12-04 15:43:46 - INFO -    Clé temporaire générée : 9ff4bb56b2db5690...
```

```
(tronquée)
2025-12-04 15:43:46 - WARNING - ⚠️ Sauvegarde de la clé secrète dans un fichier
2025-12-04 15:43:46 - WARNING -    Fichier : ./cle_secrete_anstat.txt
2025-12-04 15:43:46 - WARNING -    PROTÉGEZ ce fichier et NE LE PARTAGEZ JAMAIS
2025-12-04 15:43:46 - INFO -    ✓ Clé sauvegardée avec permissions restrictives
(400)
2025-12-04 15:43:46 - INFO -    ✓ Clé secrète chargée (longueur: 64 caractères)
2025-12-04 15:43:46 - INFO -
🔐 ÉTAPE 3 : GÉNÉRATION DES ID_ANSTAT
2025-12-04 15:43:46 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 15:43:46 - INFO -    Méthode : HMAC-SHA256
2025-12-04 15:43:46 - INFO -    Propriétés :
2025-12-04 15:43:46 - INFO -      - Déterministe (même entrée → même sortie)
2025-12-04 15:43:46 - INFO -      - Unidirectionnel (impossible de retrouver le
NUMERO_CNPS)
2025-12-04 15:43:46 - INFO -      - Sécurisé (nécessite la clé secrète)
2025-12-04 15:43:47 - INFO -    ✓ 502,000 ID_ANSTAT générés
2025-12-04 15:43:47 - INFO -
   Vérifications :
2025-12-04 15:43:48 - INFO -      - ID_ANSTAT uniques : 502,000
2025-12-04 15:43:48 - INFO -      ☑ Tous les ID_ANSTAT sont uniques
2025-12-04 15:43:48 - INFO -
   Exemples (5 premiers) :
2025-12-04 15:43:48 - INFO -      175070409808 →
e154d3c4f5241a2831617ba6e1a621cd3806730df4b82e1bbb08a73e3b0a1a97
2025-12-04 15:43:48 - INFO -      190060006564 →
37cf9d3f5c72ad3c2b835f0edcfd73107b85fe494b860697577957ed934db856
2025-12-04 15:43:48 - INFO -      196020222089 →
37a2593af677f92d80b2ce3709c8ac1a10fa406780517c7dae803c0ea72a05ba
2025-12-04 15:43:48 - INFO -      179090307526 →
93e1870d77276227ad2e7bd688290f31798d618ee71525d78bc18055aee03afb
2025-12-04 15:43:48 - INFO -      100010133300 →
83cc401e8d4a7974533dc5298b59d9b7d260eaadde0a0a3eb8c12540f45c48c5
2025-12-04 15:43:48 - INFO -
💾 ÉTAPE 4 : SAUVEGARDE
2025-12-04 15:43:48 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 15:43:48 - INFO - 💾 Sauvegarde EXCEL :
./CNPS_FUSION/NUMERO_CNPS_UNIQUES_AVEC_ID_ANSTAT.xlsx
2025-12-04 15:44:12 - INFO -    ☑ Excel sauvegardé : 27.47 MB
2025-12-04 15:44:12 - INFO - 💾 Sauvegarde CSV :
./CNPS_FUSION/NUMERO_CNPS_UNIQUES_AVEC_ID_ANSTAT.csv
2025-12-04 15:44:13 - INFO -    ☑ CSV sauvegardé : 37.34 MB
2025-12-04 15:44:13 - INFO -
   📂 Fichiers créés :
2025-12-04 15:44:13 - INFO -       - XLSX  :
./CNPS_FUSION/NUMERO_CNPS_UNIQUES_AVEC_ID_ANSTAT.xlsx
2025-12-04 15:44:13 - INFO -       - CSV   :
./CNPS_FUSION/NUMERO_CNPS_UNIQUES_AVEC_ID_ANSTAT.csv
2025-12-04 15:44:13 - INFO -
```

```
================================================================================
2025-12-04 15:44:13 - INFO - ✅ GÉNÉRATION ID_ANSTAT TERMINÉE AVEC SUCCÈS
2025-12-04 15:44:13 - INFO - ⏱️  Temps d'exécution : 40.53 secondes
2025-12-04 15:44:13 - INFO - 📊 Résultat :
2025-12-04 15:44:13 - INFO -    - NUMERO_CNPS traités  : 502,000
2025-12-04 15:44:13 - INFO -    - ID_ANSTAT générés    : 502,000
2025-12-04 15:44:13 - INFO -    - ID_ANSTAT uniques    : 502,000
2025-12-04 15:44:13 - INFO -    - Méthode              : HMAC-SHA256
2025-12-04 15:44:13 - INFO -
================================================================================
2025-12-04 15:44:13 - WARNING -
⚠️ RAPPEL SÉCURITÉ :
2025-12-04 15:44:13 - WARNING -    Une clé temporaire a été générée pour ce test.
2025-12-04 15:44:13 - WARNING -    En production, utilisez une clé secrète fixe et
sécurisée.
2025-12-04 15:44:13 - WARNING -    Clé sauvegardée dans : ./cle_secrete_anstat.txt
2025-12-04 15:44:13 - WARNING -    PROTÉGEZ ce fichier et ne le partagez JAMAIS.
```

Etape 4 :Joindre les ID_ANSTAT à la base concaténée, puis re-splitter par mois

🎯 Objectif

Ce script prend la **base concaténée CNPS 2024–2025** et la **table de mapping NUMERO_CNPS → ID_ANSTAT**, puis :

1. fait une **jointure** pour ajouter `ID_ANSTAT` à chaque ligne,
2. **supprime éventuellement** `NUMERO_CNPS` (pseudonymisation),
3. sauvegarde une **base enrichie globale** (Parquet + CSV),
4. re-split la base en **fichiers mensuels** par `PERIODE`,
5. fournit des outils de **vérification de correspondance** et d'**échantillonnage**.

🧩 Les différentes étapes

---

1. Chargement des données

`charger_fichier_concatene()`

- Cherche d'abord `TRAVAILLEURS_CONCATENES_2024_2025.parquet` dans `./CNPS_FUSION`.
- Sinon, bascule sur `TRAVAILLEURS_CONCATENES_2024_2025.csv`.

- Charge la base concaténée CNPS (tous les mois) et loggue :
  - nombre de lignes,
  - nombre de colonnes.

Rôle : récupérer la base de travail principale.

`charger_table_ids()`

- Charge la table `NUMERO_CNPS_UNIQUES_AVEC_ID_ANSTAT` :
  - en priorité `.xlsx`,
  - sinon `.csv`.
- Vérifie qu'elle contient les correspondances `NUMERO_CNPS → ID_ANSTAT`.

Rôle : fournir la table de mapping pour la pseudonymisation.

---

2. Sauvegarde des résultats

`sauvegarder_fichier_enrichi(df, nom_base, dossier)`

- Sauvegarde la base enrichie (avec `ID_ANSTAT`) en 2 formats :
  - **Parquet** : pour les gros volumes (format recommandé d'analyse),
  - **CSV** : pour compatibilité générale.
- Retourne les chemins des fichiers créés.

Rôle : produire une version complète, pseudonymisée, prête pour les analyses.

`sauvegarder_fichier_mensuel(df_mois, periode, dossier)`

- Sauvegarde chaque mois séparément dans `./CNPS_FUSION/MENSUELS_ID_ANSTAT` :
  - si ≤ 1 048 576 lignes → Excel `TRAVAILLEURS_YYYY_MM_ID_ANSTAT.xlsx`,
  - sinon → Parquet `TRAVAILLEURS_YYYY_MM_ID_ANSTAT.parquet`.
- Loggue la taille et le nombre de lignes.

Rôle : générer des fichiers mensuels pseudonymisés, exploitables par les équipes.

---

3. Pipeline principal `main(...)`

```
df_full = main(
    conserver_numero_cnps=False,
    generer_fichiers_mensuels=True
)
```

Étapes internes

1. **Charge la base concaténée** (`charger_fichier_concatene`).
2. Vérifie la présence de `NUMERO_CNPS` et `PERIODE` (sinon, pas de re-split).
3. **Charge la table de mapping** (`charger_table_ids`).
4. **Jointure LEFT** sur `NUMERO_CNPS` :
   - garde toutes les lignes de la base concaténée,
   - ajoute `ID_ANSTAT`,
   - contrôle combien de lignes n'ont pas trouvé d'ID.
5. Si `conserver_numero_cnps=False` :
   - **supprime `NUMERO_CNPS`** pour des raisons de sécurité.
6. **Sauvegarde la base enrichie** en Parquet + CSV.
7. Si `generer_fichiers_mensuels=True` et `PERIODE` présente :
   - boucle sur chaque `PERIODE`,
   - filtre `df_full` sur le mois,
   - sauvegarde le fichier mensuel correspondant.

Rôle : orchestrer toute la chaîne : jointure → pseudonymisation → export global → export mensuel.

---

4. Fonctions utilitaires

`verifier_correspondances(df_concat, df_ids)`

- Compare les `NUMERO_CNPS` présents :
  - dans la base concaténée,
  - dans la table d'IDs.
- Calcule :
  - combien sont communs,
  - combien manquent dans la table d'IDs,
  - combien sont en surplus côté IDs.
- Loggue quelques exemples de manquants.

Rôle : vérifier que la table d'IDs couvre bien tous les CNPS de la base.

`extraire_echantillon(n=1000, periode=None)`

- Charge le Parquet enrichi `TRAVAILLEURS_CONCATENES_2024_2025_ID_ANSTAT.parquet`.
- Optionnellement filtre une période (`PERIODE="2024-06"` par exemple).

- Tire un échantillon aléatoire de taille ≤ `n`.

Rôle : extraire un sous-ensemble pour contrôle qualité, debug, ou tests.

---

5. Point d'entrée

```
if __name__ == "__main__":
    df_full = main(
        conserver_numero_cnps=False,      # supprime la colonne CNPS (sécurité)
        generer_fichiers_mensuels=True    # produit les fichiers mensuels
pseudonymisés
    )
```

En résumé : ce script fait la **jointure entre CNPS et ID_ANSTAT**, pseudonymise la base, la sauvegarde, puis **recrée des fichiers mensuels** sécurisés prêts pour la diffusion interne ou l'analyse statistique.

```
import pandas as pd
import os
import logging
from pathlib import Path
from typing import Optional

# ==============================================================================
# CONFIGURATION ET LOGGING
# ==============================================================================

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)
logger = logging.getLogger(__name__)

# Configuration des chemins
DOSSIER_BASE = "./CNPS_FUSION"
DOSSIER_MENSUELS_ID = "./CNPS_FUSION/MENSUELS_ID_ANSTAT"

# Fichiers sources
FICHIER_CONCATENE_PARQUET = "TRAVAILLEURS_CONCATENES_2024_2025.parquet"
FICHIER_CONCATENE_CSV = "TRAVAILLEURS_CONCATENES_2024_2025.csv"
FICHIER_IDS = "NUMERO_CNPS_UNIQUES_AVEC_ID_ANSTAT.xlsx"

# Fichier de sortie
FICHIER_SORTIE_ENRICHI = "TRAVAILLEURS_CONCATENES_2024_2025_ID_ANSTAT"

# ==============================================================================
# FONCTIONS I/O
# ==============================================================================
```

```
def charger_fichier_concatene(dossier: str = DOSSIER_BASE) -> pd.DataFrame:
    """
    Charge le fichier concaténé (préférence PARQUET si disponible)

    Args:
        dossier: Dossier contenant le fichier

    Returns:
        DataFrame concaténé
    """
    logger.info("📥 Chargement du fichier concaténé...")

    # Essayer PARQUET d'abord (plus rapide)
    chemin_parquet = os.path.join(dossier, FICHIER_CONCATENE_PARQUET)
    chemin_csv = os.path.join(dossier, FICHIER_CONCATENE_CSV)

    if Path(chemin_parquet).exists():
        logger.info(f"   Format : PARQUET (recommandé)")
        logger.info(f"   Fichier : {chemin_parquet}")
        df = pd.read_parquet(chemin_parquet)
        logger.info(f"   ✓ {df.shape[0]:,} lignes × {df.shape[1]} colonnes")
        return df

    elif Path(chemin_csv).exists():
        logger.info(f"   Format : CSV")
        logger.info(f"   Fichier : {chemin_csv}")
        logger.info("   ⏱  Chargement CSV en cours (peut prendre quelques
minutes)...")
        df = pd.read_csv(chemin_csv)
        logger.info(f"   ✓ {df.shape[0]:,} lignes × {df.shape[1]} colonnes")
        return df

    else:
        raise FileNotFoundError(
            f"Aucun fichier concaténé trouvé dans {dossier}\n"
            f"Fichiers attendus : {FICHIER_CONCATENE_PARQUET} ou
{FICHIER_CONCATENE_CSV}\n"
            "💡 Exécutez d'abord le script de concaténation"
        )


def charger_table_ids(dossier: str = DOSSIER_BASE) -> pd.DataFrame:
    """
    Charge la table NUMERO_CNPS → ID_ANSTAT

    Args:
        dossier: Dossier contenant le fichier

    Returns:
        DataFrame avec NUMERO_CNPS et ID_ANSTAT
    """
    chemin_xlsx = os.path.join(dossier, FICHIER_IDS)
    chemin_csv = chemin_xlsx.replace('.xlsx', '.csv')
```

```
    logger.info("📥 Chargement de la table de mapping...")

    if Path(chemin_xlsx).exists():
        logger.info(f"   Fichier : {chemin_xlsx}")
        df = pd.read_excel(chemin_xlsx, engine='openpyxl')
    elif Path(chemin_csv).exists():
        logger.info(f"   Fichier : {chemin_csv}")
        df = pd.read_csv(chemin_csv)
    else:
        raise FileNotFoundError(
            f"Fichier de mapping introuvable : {chemin_xlsx}\n"
            "💡 Exécutez d'abord le script de génération ID_ANSTAT"
        )

    logger.info(f"   ✓ {len(df):,} correspondances NUMERO_CNPS → ID_ANSTAT")

    return df


def sauvegarder_fichier_enrichi(
    df: pd.DataFrame,
    nom_base: str = FICHIER_SORTIE_ENRICHI,
    dossier: str = DOSSIER_BASE
) -> dict:
    """
    Sauvegarde le fichier enrichi en plusieurs formats

    Args:
        df: DataFrame enrichi
        nom_base: Nom de base du fichier
        dossier: Dossier de destination

    Returns:
        dict: Chemins des fichiers créés
    """
    Path(dossier).mkdir(parents=True, exist_ok=True)
    fichiers = {}

    logger.info(f"💾 Sauvegarde du fichier enrichi ({len(df):,} lignes)...")

    # PARQUET (recommandé pour gros fichiers)
    chemin_parquet = os.path.join(dossier, f"{nom_base}.parquet")
    logger.info(f"   Format : PARQUET")
    df.to_parquet(chemin_parquet, index=False, compression='snappy')
    taille = Path(chemin_parquet).stat().st_size / (1024 * 1024)
    logger.info(f"   ✅ Sauvegardé : {chemin_parquet} ({taille:.2f} MB)")
    fichiers['parquet'] = chemin_parquet

    # CSV (pour compatibilité)
    chemin_csv = os.path.join(dossier, f"{nom_base}.csv")
    logger.info(f"   Format : CSV")
    df.to_csv(chemin_csv, index=False, encoding='utf-8-sig')
    taille = Path(chemin_csv).stat().st_size / (1024 * 1024)
```

```
    logger.info(f"   ✅ Sauvegardé : {chemin_csv} ({taille:.2f} MB)")
    fichiers['csv'] = chemin_csv

    return fichiers


def sauvegarder_fichier_mensuel(
    df_mois: pd.DataFrame,
    periode: str,
    dossier: str = DOSSIER_MENSUELS_ID
) -> str:
    """
    Sauvegarde un fichier mensuel

    Args:
        df_mois: DataFrame du mois
        periode: Période au format YYYY-MM
        dossier: Dossier de destination

    Returns:
        Chemin du fichier créé
    """
    Path(dossier).mkdir(parents=True, exist_ok=True)

    annee, mois = periode.split("-")
    nom_fichier = f"TRAVAILLEURS_{annee}_{mois}_ID_ANSTAT.xlsx"
    chemin = os.path.join(dossier, nom_fichier)

    # Vérifier si Excel peut gérer la taille
    EXCEL_MAX_ROWS = 1_048_576

    if len(df_mois) > EXCEL_MAX_ROWS:
        logger.warning(f"   ⚠ {len(df_mois):,} lignes > limite Excel
({EXCEL_MAX_ROWS:,})")
        logger.info(f"   → Sauvegarde en PARQUET au lieu d'Excel")

        nom_fichier_parquet = nom_fichier.replace('.xlsx', '.parquet')
        chemin = os.path.join(dossier, nom_fichier_parquet)
        df_mois.to_parquet(chemin, index=False, compression='snappy')
    else:
        df_mois.to_excel(chemin, index=False, engine='openpyxl')

    taille = Path(chemin).stat().st_size / (1024 * 1024)
    logger.info(f"   ✅ {periode} : {len(df_mois):,} lignes → {Path(chemin).name}
({taille:.2f} MB)")

    return chemin


# ============================================================================
# FONCTION PRINCIPALE
# ============================================================================

def main(
```

```
    conserver_numero_cnps: bool = False,
    generer_fichiers_mensuels: bool = True
) -> Optional[pd.DataFrame]:
    """
    Pipeline principal de jointure et re-split

    Args:
        conserver_numero_cnps: Si False, supprime NUMERO_CNPS après jointure
(recommandé)
        generer_fichiers_mensuels: Si True, génère les fichiers mensuels

    Returns:
        DataFrame enrichi avec ID_ANSTAT
    """
    import time
    start_time = time.time()

    logger.info("=" * 80)
    logger.info("🚀 JOINTURE ID_ANSTAT + RE-SPLIT PAR MOIS (VERSION LOCALE)")
    logger.info("=" * 80)

    try:
        # ÉTAPE 1: Chargement de la base concaténée
        logger.info("\n📥 ÉTAPE 1 : CHARGEMENT DE LA BASE CONCATÉNÉE")
        logger.info("-" * 80)
        df_concat = charger_fichier_concatene()

        # Vérifications
        if "NUMERO_CNPS" not in df_concat.columns:
            raise KeyError("La colonne 'NUMERO_CNPS' est absente de la base
concaténée.")

        if "PERIODE" not in df_concat.columns:
            logger.warning("⚠️ La colonne 'PERIODE' est absente.")
            logger.warning("   Le re-split par mois sera impossible.")
            generer_fichiers_mensuels = False

        # ÉTAPE 2: Chargement de la table de mapping
        logger.info("\n📥 ÉTAPE 2 : CHARGEMENT DE LA TABLE DE MAPPING NUMERO_CNPS
→ ID_ANSTAT")
        logger.info("-" * 80)
        df_ids = charger_table_ids()

        if "NUMERO_CNPS" not in df_ids.columns or "ID_ANSTAT" not in
df_ids.columns:
            raise KeyError(
                "Le fichier de mapping doit contenir les colonnes "
                "'NUMERO_CNPS' et 'ID_ANSTAT'."
            )

        # ÉTAPE 3: Jointure
        logger.info("\n🔗 ÉTAPE 3 : JOINTURE SUR 'NUMERO_CNPS'")
        logger.info("-" * 80)
        logger.info("   Type de jointure : LEFT JOIN (toutes les lignes
```

```
conservées)")
        logger.info("   Fusion en cours...")

        df_full = df_concat.merge(
            df_ids[["NUMERO_CNPS", "ID_ANSTAT"]],
            on="NUMERO_CNPS",
            how="left",
            validate="m:1"  # chaque NUMERO_CNPS → au plus 1 ID_ANSTAT
        )

        # Contrôles
        nb_total = len(df_full)
        nb_sans_id = df_full["ID_ANSTAT"].isna().sum()

        logger.info(f"   ✓ Base enrichie : {nb_total:,} lignes")
        logger.info(f"   → Lignes avec ID_ANSTAT    : {nb_total - nb_sans_id:,}")
        logger.info(f"   → Lignes sans ID_ANSTAT    : {nb_sans_id:,}")

        if nb_sans_id > 0:
            pct_sans_id = 100 * nb_sans_id / nb_total
            logger.warning(f"   ⚠ {pct_sans_id:.2f}% des lignes n'ont pas de
correspondance")
            logger.warning("      Vérifiez que tous les NUMERO_CNPS étaient dans
la table des uniques")
        else:
            logger.info("   ✅ Toutes les lignes ont un ID_ANSTAT")

        # Option : supprimer NUMERO_CNPS pour plus de sécurité
        if not conserver_numero_cnps:
            logger.info("\n🔒 SÉCURITÉ : Suppression de la colonne NUMERO_CNPS")
            logger.info("   (Pour éviter tout lien avec les données
personnelles)")
            df_full = df_full.drop(columns=["NUMERO_CNPS"])
            logger.info("   ✅ Colonne NUMERO_CNPS supprimée")

        # ÉTAPE 4: Sauvegarde de la base enrichie
        logger.info("\n💾 ÉTAPE 4 : SAUVEGARDE DE LA BASE CONCATÉNÉE ENRICHIE")
        logger.info("-" * 80)
        fichiers = sauvegarder_fichier_enrichi(df_full)

        logger.info(f"\n   📂 Fichiers créés :")
        for format_type, chemin in fichiers.items():
            logger.info(f"      - {format_type.upper():7s} : {chemin}")

        # ÉTAPE 5: Re-split par mois
        if generer_fichiers_mensuels and "PERIODE" in df_full.columns:
            logger.info("\n📅 ÉTAPE 5 : RE-SPLIT PAR MOIS SELON 'PERIODE'")
            logger.info("-" * 80)

            periodes = sorted(df_full["PERIODE"].dropna().unique())
            logger.info(f"   Périodes trouvées : {len(periodes)}")
            logger.info(f"   De {periodes[0]} à {periodes[-1]}")

            logger.info(f"\n   Génération des {len(periodes)} fichiers
```

```
mensuels...")

            for i, periode in enumerate(periodes, 1):
                df_mois = df_full[df_full["PERIODE"] == periode].copy()
                logger.info(f"\n   [{i}/{len(periodes)}] Période {periode}:")
                sauvegarder_fichier_mensuel(df_mois, periode)

            logger.info(f"\n   ✅ {len(periodes)} fichiers mensuels créés dans :
{DOSSIER_MENSUELS_ID}/")

        # Résumé final
        elapsed_time = time.time() - start_time
        logger.info("\n" + "=" * 80)
        logger.info("✅ JOINTURE ET RE-SPLIT TERMINÉS AVEC SUCCÈS")
        logger.info(f"⏱️  Temps d'exécution : {elapsed_time:.2f} secondes
({elapsed_time/60:.1f} min)")
        logger.info(f"📊 Résultat :")
        logger.info(f"   - Lignes traitées       : {len(df_full):,}")
        logger.info(f"   - Lignes avec ID_ANSTAT :
{(~df_full['ID_ANSTAT'].isna()).sum():,}")

        if generer_fichiers_mensuels and "PERIODE" in df_full.columns:
            logger.info(f"   - Fichiers mensuels     : {len(periodes)}")
            logger.info(f"   - Localisation          : {DOSSIER_MENSUELS_ID}/")

        logger.info(f"\n   📂 Base enrichie disponible :")
        logger.info(f"      - PARQUET : {fichiers['parquet']}")
        logger.info(f"      - CSV     : {fichiers['csv']}")

        logger.info("=" * 80)

        # Avertissement sécurité
        if not conserver_numero_cnps:
            logger.info("\n🔒 SÉCURITÉ :")
            logger.info("   ✅ NUMERO_CNPS supprimé des fichiers finaux")
            logger.info("   ✅ Seul ID_ANSTAT est présent (pseudonymisation
complète)")
        else:
            logger.warning("\n⚠️ SÉCURITÉ :")
            logger.warning("   NUMERO_CNPS conservé dans les fichiers")
            logger.warning("   Recommandation : supprimer après vérifications")

        return df_full

    except FileNotFoundError as e:
        logger.error(f"\n❌ FICHIER INTROUVABLE: {e}")
        return None

    except Exception as e:
        logger.error(f"\n❌ ERREUR CRITIQUE: {e}", exc_info=True)
        raise


# ==============================================================================
```

```python
# FONCTIONS UTILITAIRES
# ==============================================================================

def verifier_correspondances(
    df_concat: pd.DataFrame,
    df_ids: pd.DataFrame
) -> dict:
    """
    Vérifie les correspondances entre les deux tables

    Args:
        df_concat: DataFrame concaténé
        df_ids: DataFrame des IDs

    Returns:
        dict: Statistiques de correspondance
    """
    logger.info("🔍 Vérification des correspondances...")

    cnps_concat = set(df_concat["NUMERO_CNPS"].unique())
    cnps_ids = set(df_ids["NUMERO_CNPS"].unique())

    communs = cnps_concat & cnps_ids
    manquants = cnps_concat - cnps_ids
    surplus = cnps_ids - cnps_concat

    stats = {
        'cnps_concat': len(cnps_concat),
        'cnps_ids': len(cnps_ids),
        'communs': len(communs),
        'manquants_dans_ids': len(manquants),
        'surplus_dans_ids': len(surplus)
    }

    logger.info(f"\n   NUMERO_CNPS dans base concaténée :
{stats['cnps_concat']:,}")
    logger.info(f"   NUMERO_CNPS dans table IDs       : {stats['cnps_ids']:,}")
    logger.info(f"   En commun                        : {stats['communs']:,}")

    if manquants:
        logger.warning(f"   ⚠ Manquants dans table IDs      :
{len(manquants):,}")
        logger.warning(f"      Exemples : {list(manquants)[:5]}")

    if surplus:
        logger.info(f"   ℹ  Surplus dans table IDs        : {len(surplus):,}")

    return stats


def extraire_echantillon(
    n: int = 1000,
    periode: Optional[str] = None
) -> pd.DataFrame:
```

```
    """
    Extrait un échantillon de la base enrichie

    Args:
        n: Nombre de lignes à extraire
        periode: Période spécifique (optionnel)

    Returns:
        DataFrame échantillon
    """
    logger.info(f"📊 Extraction d'un échantillon de {n:,} lignes...")

    # Charger le fichier enrichi
    chemin = os.path.join(DOSSIER_BASE, f"{FICHIER_SORTIE_ENRICHI}.parquet")
    df = pd.read_parquet(chemin)

    # Filtrer par période si demandé
    if periode and "PERIODE" in df.columns:
        df = df[df["PERIODE"] == periode]
        logger.info(f"   Filtré sur période {periode}")

    # Échantillonner
    echantillon = df.sample(n=min(n, len(df)))
    logger.info(f"   ✓ Échantillon de {len(echantillon):,} lignes extrait")

    return echantillon


# ============================================================================
# POINT D'ENTRÉE
# ============================================================================

if __name__ == "__main__":
    df_full = main(
        conserver_numero_cnps=False,      # Supprime NUMERO_CNPS (sécurité)
        generer_fichiers_mensuels=True    # Génère les fichiers mensuels
    )
```

```
2025-12-04 16:04:14 - INFO -
================================================================================
2025-12-04 16:04:14 - INFO - 🚀 JOINTURE ID_ANSTAT + RE-SPLIT PAR MOIS (VERSION
LOCALE)
2025-12-04 16:04:14 - INFO -
================================================================================
2025-12-04 16:04:14 - INFO -
📥 ÉTAPE 1 : CHARGEMENT DE LA BASE CONCATÉNÉE
2025-12-04 16:04:14 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 16:04:14 - INFO - 📥 Chargement du fichier concaténé...
2025-12-04 16:04:14 - INFO -    Format : PARQUET (recommandé)
2025-12-04 16:04:14 - INFO -    Fichier :
```

```
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025.parquet
2025-12-04 16:04:15 - INFO -    ✓ 1,503,000 lignes × 14 colonnes
2025-12-04 16:04:15 - INFO -
📥 ÉTAPE 2 : CHARGEMENT DE LA TABLE DE MAPPING NUMERO_CNPS → ID_ANSTAT
2025-12-04 16:04:15 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 16:04:15 - INFO - 📥 Chargement de la table de mapping...
2025-12-04 16:04:15 - INFO -    Fichier :
./CNPS_FUSION/NUMERO_CNPS_UNIQUES_AVEC_ID_ANSTAT.xlsx
2025-12-04 16:04:34 - INFO -    ✓ 502,000 correspondances NUMERO_CNPS → ID_ANSTAT
2025-12-04 16:04:34 - INFO -
🔗 ÉTAPE 3 : JOINTURE SUR 'NUMERO_CNPS'
2025-12-04 16:04:34 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 16:04:34 - INFO -    Type de jointure : LEFT JOIN (toutes les lignes
conservées)
2025-12-04 16:04:34 - INFO -    Fusion en cours...
2025-12-04 16:04:34 - INFO -    ✓ Base enrichie : 1,503,000 lignes
2025-12-04 16:04:34 - INFO -    → Lignes avec ID_ANSTAT    : 1,503,000
2025-12-04 16:04:34 - INFO -    → Lignes sans ID_ANSTAT    : 0
2025-12-04 16:04:34 - INFO -    ☑ Toutes les lignes ont un ID_ANSTAT
2025-12-04 16:04:34 - INFO -
🔒 SÉCURITÉ : Suppression de la colonne NUMERO_CNPS
2025-12-04 16:04:34 - INFO -    (Pour éviter tout lien avec les données
personnelles)
2025-12-04 16:04:34 - INFO -    ☑ Colonne NUMERO_CNPS supprimée
2025-12-04 16:04:34 - INFO -
💾 ÉTAPE 4 : SAUVEGARDE DE LA BASE CONCATÉNÉE ENRICHIE
2025-12-04 16:04:34 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 16:04:34 - INFO - 💾 Sauvegarde du fichier enrichi (1,503,000
lignes)...
2025-12-04 16:04:34 - INFO -    Format : PARQUET
2025-12-04 16:04:38 - INFO -    ☑ Sauvegardé :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025_ID_ANSTAT.parquet (128.20 MB)
2025-12-04 16:04:38 - INFO -    Format : CSV
2025-12-04 16:04:49 - INFO -    ☑ Sauvegardé :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025_ID_ANSTAT.csv (363.83 MB)
2025-12-04 16:04:49 - INFO -
   📂 Fichiers créés :
2025-12-04 16:04:49 - INFO -       - PARQUET :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025_ID_ANSTAT.parquet
2025-12-04 16:04:49 - INFO -       - CSV     :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025_ID_ANSTAT.csv
2025-12-04 16:04:49 - INFO -
📅 ÉTAPE 5 : RE-SPLIT PAR MOIS SELON 'PERIODE'
2025-12-04 16:04:49 - INFO - ----------------------------------------------------
---------------------------
2025-12-04 16:04:49 - INFO -    Périodes trouvées : 3
2025-12-04 16:04:49 - INFO -    De 2024-01 à 2024-03
2025-12-04 16:04:49 - INFO -
```

```
   Génération des 3 fichiers mensuels...
2025-12-04 16:04:50 - INFO -
   [1/3] Période 2024-01:
2025-12-04 16:07:27 - INFO -    ☑ 2024-01 : 500,000 lignes →
TRAVAILLEURS_2024_01_ID_ANSTAT.xlsx (68.60 MB)
2025-12-04 16:07:28 - INFO -
   [2/3] Période 2024-02:
2025-12-04 16:10:03 - INFO -    ☑ 2024-02 : 501,000 lignes →
TRAVAILLEURS_2024_02_ID_ANSTAT.xlsx (68.74 MB)
2025-12-04 16:10:03 - INFO -
   [3/3] Période 2024-03:
2025-12-04 16:12:39 - INFO -    ☑ 2024-03 : 502,000 lignes →
TRAVAILLEURS_2024_03_ID_ANSTAT.xlsx (68.88 MB)
2025-12-04 16:12:39 - INFO -
   ☑ 3 fichiers mensuels créés dans : ./CNPS_FUSION/MENSUELS_ID_ANSTAT/
2025-12-04 16:12:39 - INFO -
================================================================================
2025-12-04 16:12:39 - INFO - ☑ JOINTURE ET RE-SPLIT TERMINÉS AVEC SUCCÈS
2025-12-04 16:12:39 - INFO - ⏱  Temps d'exécution : 504.36 secondes (8.4 min)
2025-12-04 16:12:39 - INFO - 📊 Résultat :
2025-12-04 16:12:39 - INFO -    - Lignes traitées       : 1,503,000
2025-12-04 16:12:39 - INFO -    - Lignes avec ID_ANSTAT : 1,503,000
2025-12-04 16:12:39 - INFO -    - Fichiers mensuels     : 3
2025-12-04 16:12:39 - INFO -    - Localisation          :
./CNPS_FUSION/MENSUELS_ID_ANSTAT/
2025-12-04 16:12:39 - INFO -
   🗁 Base enrichie disponible :
2025-12-04 16:12:39 - INFO -       - PARQUET :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025_ID_ANSTAT.parquet
2025-12-04 16:12:39 - INFO -       - CSV     :
./CNPS_FUSION/TRAVAILLEURS_CONCATENES_2024_2025_ID_ANSTAT.csv
2025-12-04 16:12:39 - INFO -
================================================================================
2025-12-04 16:12:39 - INFO -
🔒 SÉCURITÉ :
2025-12-04 16:12:39 - INFO -    ☑ NUMERO_CNPS supprimé des fichiers finaux
2025-12-04 16:12:39 - INFO -    ☑ Seul ID_ANSTAT est présent (pseudonymisation
complète)
```